

Rockchip RK356x USB Developer Guide

ID: RK-SM-YF-142

Release Version: V1.4.0

Release Date: 2024-04-24

Security Level: ☐Top-Secret ☐Secret ☐Internal ☒Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED "AS IS". ROCKCHIP ELECTRONICS CO., LTD. ("ROCKCHIP") DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS, MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip's registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2025. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian, PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

This document provides a development guide for the RK356x USB module, aimed at helping engineers understand the hardware design and software driver design of the RK356x USB controller and USB PHY, so that developers can flexibly design and rapidly develop according to the USB application requirements of the product.

Chipset	Kernel Version
RK3566, RK3568	Linux-4.19 and above

Intended Audience

This document (this guide) is mainly intended for:

Technical support engineers

Software development engineers

Revision History

Version	Author	Date	Change Description
V1.0.0	Wu Liangfeng	2021-01-18	Initial version
V1.1.0	Wu Liangfeng	2021-03-10	1. Modified document identifier according to Rockchip document writing standards 2. Modified USB 2.0 PHYs power optimization method
V1.2.0	Wu Liangfeng	2021-03-15	Added software modification instructions for USB 2.0 PHYs power optimization
V1.3.0	Wu Liangfeng	2022-09-26	1. Added hardware circuit description for Type-C USB PD 2. Added Linux-4.19/5.10 Type-C USB DTS configuration instructions 3. Added Type-C controller chip support list
V1.4.0	Wu Liangfeng	2024-04-24	Added DTS properties and instructions for USB2 only scheme

Contents

Rockchip RK356x USB Developer Guide

1. RK356x USB Controller and PHY Introduction
2. RK356x USB Hardware Circuit Design
 - 2.1 RK356x USB 2.0/3.0 Power Supply and Power Consumption Control
 - 2.1.1 RK356x USB 2.0/3.0 Controller Power Supply and Power Consumption Control
 - 2.1.2 RK356x USB 2.0 PHYs Power Supply and Power Consumption Control
 - 2.1.3 RK356x USB 3.0 PHYs Power Supply and Power Consumption Control
 - 2.2 RK3566 USB 2.0 OTG Micro-B Interface Hardware Circuit
 - 2.3 RK3566 USB 2.0 OTG Type-C Interface Hardware Circuitry
 - 2.3.1 RK3566 Type-C USB 2.0 Only Hardware Circuit Design[No PD]
 - 2.3.2 RK3566 Type-C USB 2.0 + PD Hardware Circuit
 - 2.4 RK356x USB 2.0 Host Type-A Interface Hardware Circuit
 - 2.5 RK3568 USB 3.0 OTG Type-A Interface Hardware Circuit
 - 2.6 RK3568 USB 3.0 OTG Type-C Interface Hardware Circuitry
 - 2.7 RK356x USB 3.0 HOST Type-A Interface Hardware Circuit
3. RK356x USB DTS Configuration
 - 3.1 RK3568 USB Controller and PHY DTSI Nodes
 - 3.2 RK3568 USB 3.0 OTG to USB 2.0 Only Configuration
 - 3.3 RK356x USB 3.0 Host to USB 2.0 only Configuration
 - 3.4 RK356x Type-C USB DTS Configuration
 - 3.4.1 Linux-4.19 RK3566 Type-C USB 2.0 + FUSB302 DTS Configuration
 - 3.4.2 Linux-4.19 RK3566 Type-C USB 2.0 + HUSB311 DTS Configuration
 - 3.4.3 Linux-5.10 RK3566 Type-C USB 2.0 + FUSB302 DTS Configuration
 - 3.4.4 Linux-5.10 RK3566 Type-C USB 2.0 + HUSB311 DTS Configuration
 - 3.5 USB VBUS Configuration
 - 3.6 Considerations for Linux USB DT Configuration
 - 3.6.1 Important Attributes of USB DT Explanation
 - 3.6.1.1 USB 3.1 Controller DT Properties
 - 3.6.1.2 USB2 PHY DT Properties
 - 3.6.1.3 USB 3.0/SATA/QSGMII ComboPHY DT Properties
4. RK356x USB OTG Mode Switching Commands
5. Type-C Controller Chip Support List
6. References

1. RK356x USB Controller and PHY Introduction

The RK356x USB Controller support list is shown in Table 1 below.

Table 1 RK356x USB Controller List

Chip/Controller	USB 2.0 HOST (EHCI/OHCI)	USB 3.0/2.0 OTG (DWC3/xHCI)
RK3566	2	2 (OTG 2.0 + Host 3.0)
RK3568	2	2 (OTG 3.0 + Host 3.0)

The RK356x USB PHY support list is shown in Table 2 below.

Table 2 RK356x USB PHY List

Chip/PHY	USB 2.0 ComboPHY	USB 3.0/SATA/QSGMII ComboPHY
RK3566	2 [2 × port]	1
RK3568	2 [2 × port]	2

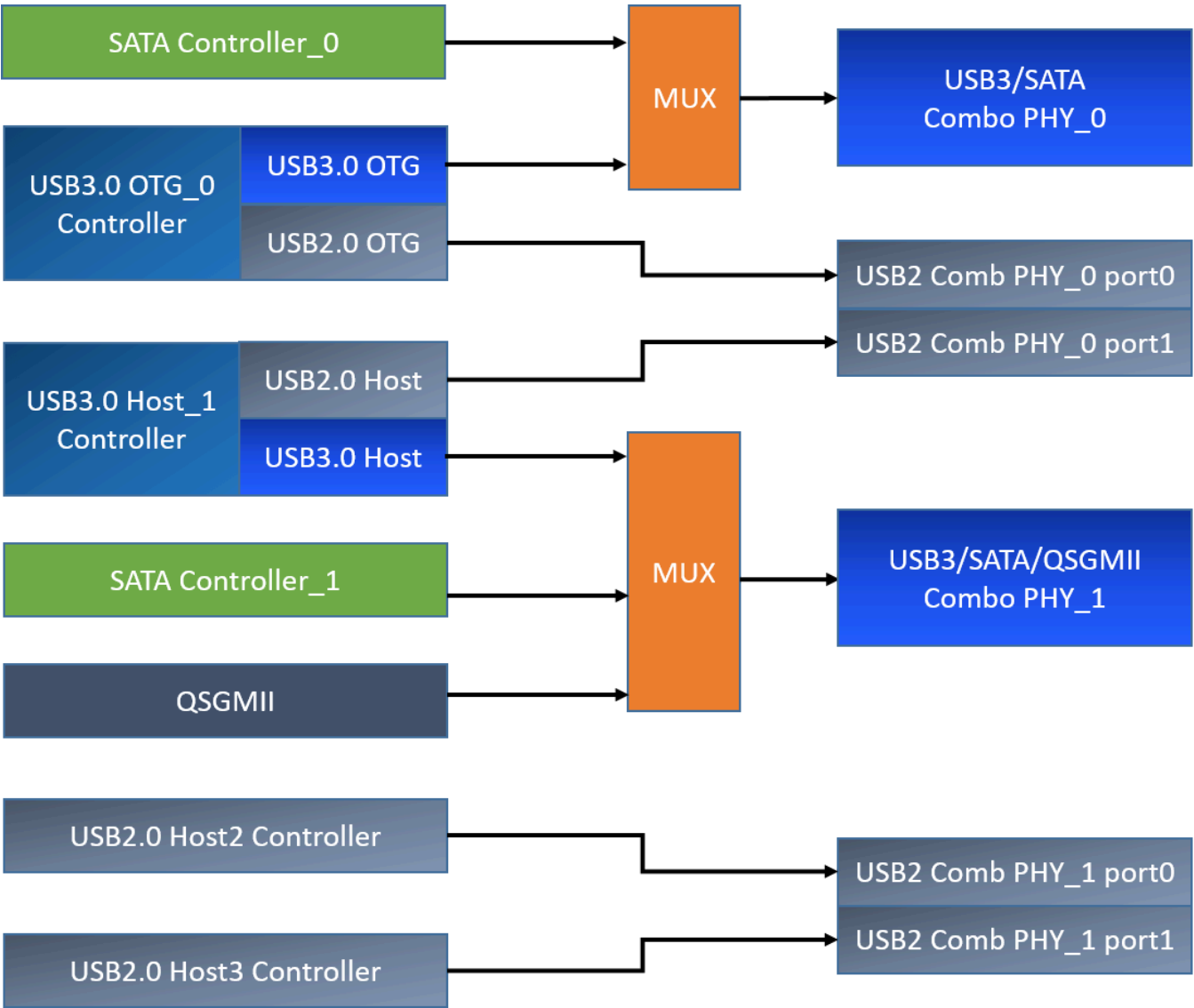
Note:

1. In the table, the number N indicates support for N independent USB controllers;
2. In the table, [2 × ports] indicates that one PHY supports two USB ports;
3. In the table, "EHCI/OHCI" indicates that the USB controller integrates an EHCI controller and an OHCI controller; "DWC3/xHCI" indicates that the USB controller integrates a DWC3 controller and an xHCI controller;
4. The DWC3 OTG controller of RK3566 only supports OTG 2.0 and does not support OTG 3.0, meaning it can only support up to USB 2.0 480Mbps transfer;

The difference between the USB modules of RK3566 and RK3568 is that RK3568 supports OTG 3.0, while RK3566 only supports OTG 2.0, but both support 1 USB 3.0 Host and 2 USB 2.0 Hosts. As shown in Figure 1-1, which is a connection diagram of RK3568 USB controllers and PHYs. From Figure 1-1, it can be seen that:

1. The USB 3.0 OTG controller is multiplexed with the SATA_0 controller using the USB3/SATA Combo PHY_0;
2. The USB 3.0 Host_1 controller is multiplexed with the SATA_1/QSGMII controller using the USB3/SATA/QSGMII Combo PHY_1;
3. The USB 3.0 OTG controller and the USB 3.0 Host_1 controller use port0 and port1 of the USB 2.0 Comb PHY_0, respectively;
4. The USB 2.0 Host_2 controller and the USB 2.0 Host_3 controller use port0 and port1 of the USB 2.0 Comb PHY_1, respectively;

It should be noted that USB3/SATA Combo PHY_0 and USB3/SATA/QSGMII Combo PHY_1 can only support one working mode at the same time, meaning that USB3 and SATA, QSGMII interfaces are mutually exclusive. Developers can flexibly configure the kernel DTS according to the product form to enable the corresponding peripheral interfaces. For the configuration method of USB DTS, please refer to [RK356x USB DTS Configuration](#).



RK3568 USB2.0/3.0 Controllers and PHYs hardware framework

Figure 1-1 Connection diagram of RK3568 USB controllers and PHYs

2. RK356x USB Hardware Circuit Design

This section primarily illustrates the various hardware circuit design schemes that RK356x USB can support in practical applications. As shown in Figure 2-1 below, the RK3568 USB interface diagram indicates that RK3568 USB supports a total of 4 USB peripheral interfaces, including 1 USB 3.0 OTG interface, 1 USB 3.0 Host interface, and 2 USB 2.0 Host interfaces. Similarly, RK3566 USB also supports 4 USB peripheral interfaces, including 1 USB 2.0 OTG interface, 1 USB 3.0 Host interface, and 2 USB 2.0 Host interfaces.

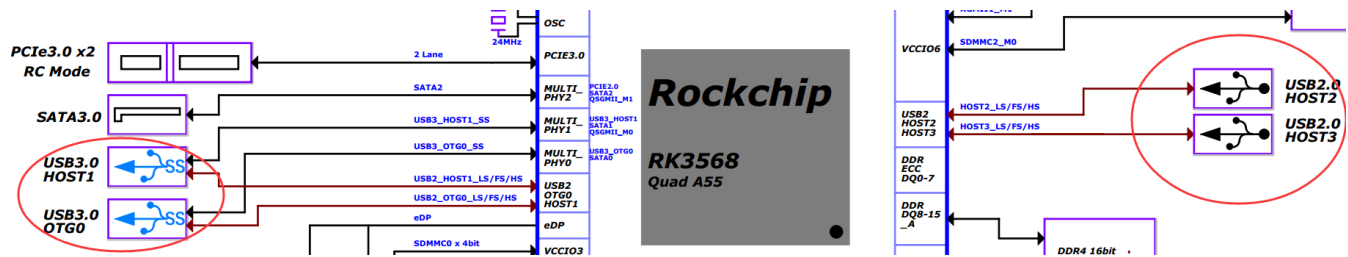


Figure 2-1 Schematic diagram of RK3568 USB peripheral interfaces

2.1 RK356x USB 2.0/3.0 Power Supply and Power Consumption Control

Principles of power consumption control are as follows:

1. In hardware circuit design, if it is possible to directly disconnect the power supply of USB 2.0 PHY or Combo PHY, then the PHY power consumption is 0, and the corresponding DTS node configuration should be set to disable;
2. In hardware circuit design, if it is necessary to ensure the power supply of USB 2.0 PHY, **then in DTS, both port nodes corresponding to the PHY should be configured as enable**, so that the PHY driver can set unused ports to enter suspend mode;
3. For unused USB controllers, the corresponding controller DTS node should be configured as disable;
4. For unused Combo PHYs, the corresponding Combo PHY DTS node should be configured as disable.

2.1.1 RK356x USB 2.0/3.0 Controller Power Supply and Power Consumption Control

The power supply for the RK356x USB 2.0/3.0 controller is VDD_LOGIC, and the USB 3.0 controller shares the internal power domain PD_PIPE with PCIE/SATA/XPCS, which can dynamically control the switch of PD_PIPE based on the working condition of the USB 3.0 interface to reduce the power consumption of the USB 3.0 controller. However, the USB 2.0 controller does not have a power domain.

It should be noted that the RK356x Linux kernel already supports the dynamic switch of USB 3.0 OTG's PD_PIPE and supports the USB Host auto suspend function (referring to the USB Host controller automatically entering a suspend low-power state when no external devices are connected to the USB Host interface), thus developers do not need to debug the power consumption control of the USB 2.0/3.0 controllers.

If all USB interfaces are not required on the product, it is recommended to configure the DTS of the unused USB controllers to disable in the kernel to reduce the power consumption of the USB controllers. The following is the method to disable USB 2.0 Host_2 and USB 2.0 Host_3:

```
&usb_host0_ehci {
    status = "disabled";
};

&usb_host0_ohci {
    status = "disabled";
};
```

```

&usb_host1_ehci {
    status = "disabled";
};

&usb_host1_ohci {
    status = "disabled";
};

```

2.1.2 RK356x USB 2.0 PHYs Power Supply and Power Consumption Control

RK356x includes 2 USB 2.0 Combo PHYs. Specifically, USB OTG and USB Host_1 utilize the USB 2.0 Combo PHY_0; whereas USB Host_2 and USB Host_3 use the USB 2.0 Combo PHY_1. Each USB 2.0 Combo PHY is powered by three voltage rails: 3.3V, 1.8V, and 0.9V, as shown in Table 3 below.

It is important to note that in actual circuitry, voltage values exceeding the specified maximum or falling below the specified minimum (with a voltage tolerance of $\pm 10\%$) may lead to abnormal USB connections.

Table 3 USB 2.0 PHY Power Supplies

Supply Voltage	Min	Typ	Max	Unit
USB_AVDD_3V3	3.0	3.3	3.6	V
USB_AVDD_1V8	1.62	1.8	1.98	V
USB_AVDD_0V9	0.81	0.9	0.99	V

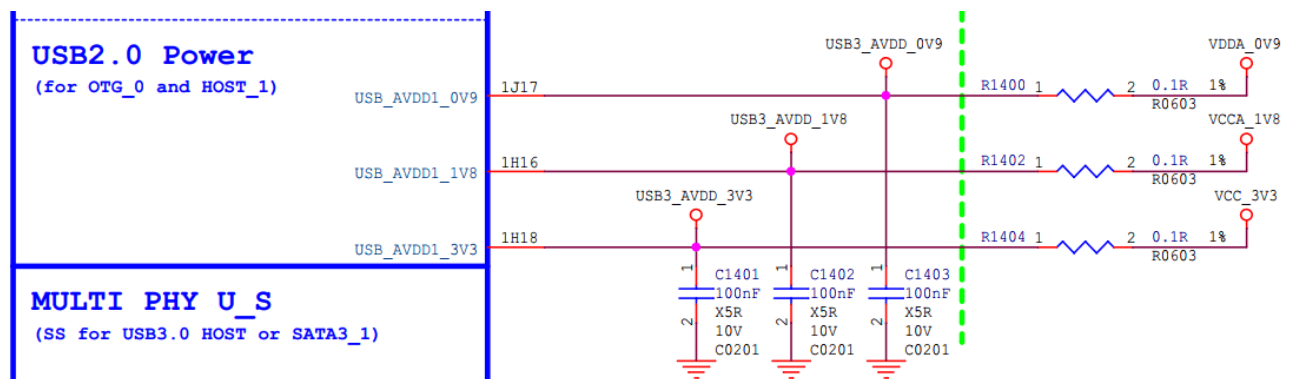


Figure 2-2 USB 2.0 Combo PHY_0 (for OTG and Host_1) Power Supply Circuit

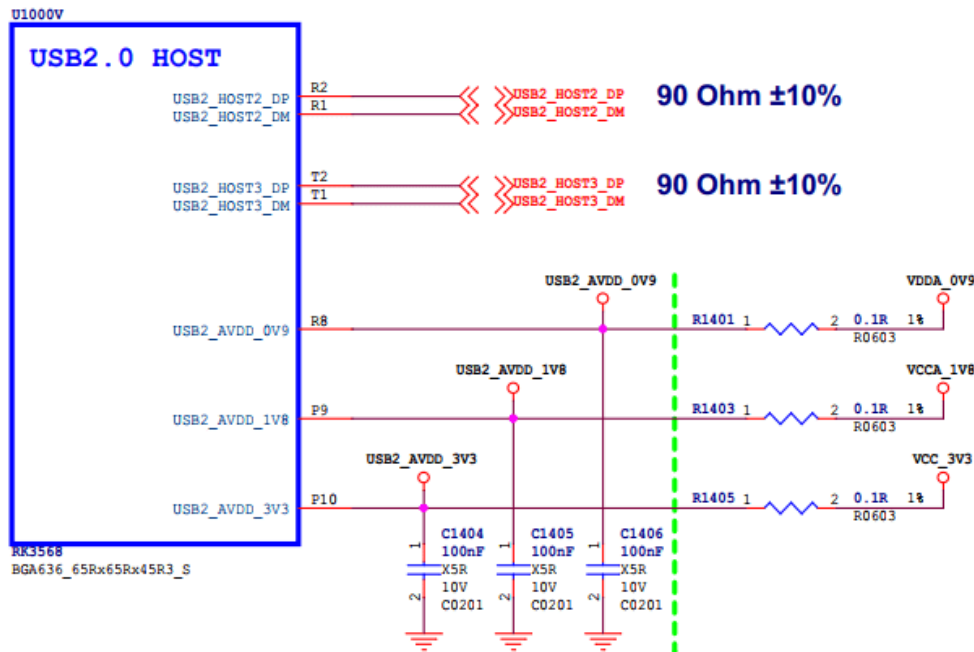


Figure 2-3 USB 2.0 Combo PHY_1 (for Host_2 and Host_3) Power Supply Circuit

After power-up, the USB 2.0 Combo PHYs are configured to operate normally by default, hence dynamic power consumption control within the USB PHY driver is necessary (supported by the PHY driver code), or the power supply to the USB PHYs should be disconnected. It should be noted that **USB 2.0 Combo PHY_0 is used for USB OTG, as it must support USB firmware downloading at a minimum, thus the power supply to USB 2.0 Combo PHY_0 must not be interrupted.**

In actual RK356x products, a common scenario for USB 2.0 PHY power consumption optimization is as follows:

The product only needs to support the USB OTG function and does not require USB 3.0 Host_1/USB 2.0 Host_2/USB 2.0 Host_3. It is recommended to optimize the USB 2.0 PHY power consumption as follows:

1. Disable the u2phy0_host node in the kernel DTS

The purpose is to prevent the initialization of u2phy0_host during the probe process of the PHY driver, which would increase the power consumption of u2phy0_host.
2. Disconnect the power supply of USB 2.0 Combo PHY_1

As shown in Figure 2-4 of the hardware circuit, ground the three power rails of Combo PHY_1.

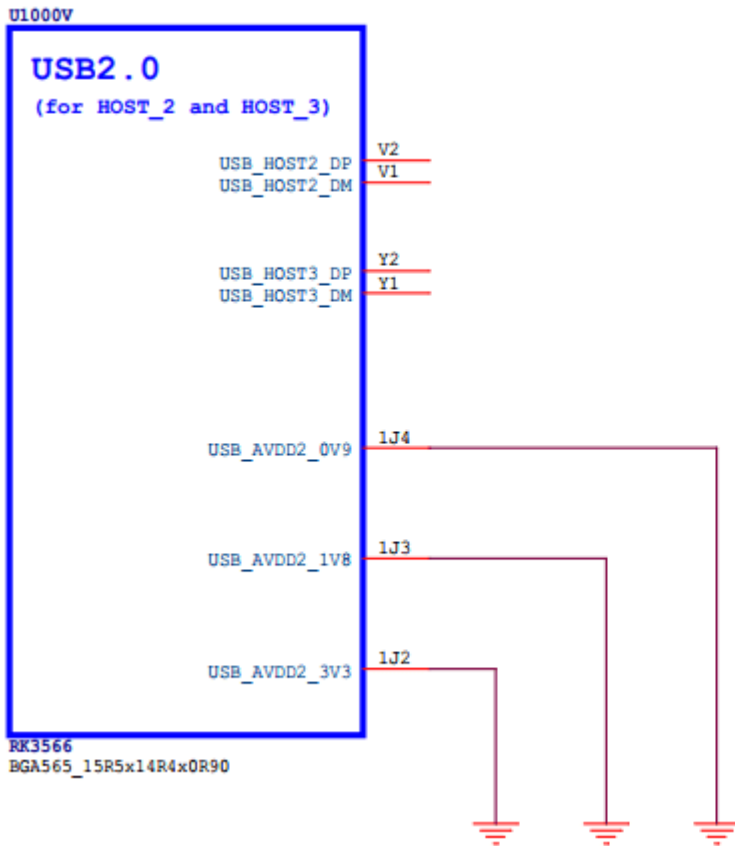


Figure 2-4 Disconnecting Power Supply Circuit for USB 2.0 Combo PHY_1 (for Host_2 and Host_3)

Since Combo PHY_1 corresponds to a USB 2.0 Host2 and USB 2.0 Host3, it is necessary to modify the kernel DTS accordingly to disable the PHY nodes `u2phy1_host` and `u2phy1_otg`, as well as the corresponding USB controller nodes `usb_host1_ehci` and `usb_host1_ohci` in the kernel DTS, as shown below:

```
&usb_host1_ehci {
    status = "disabled";
};

&usb_host1_ohci {
    status = "disabled";
};

&u2phy1_host {
    status = "disabled";
};

&u2phy1_otg {
    status = "disabled";
};
```

It is particularly important to note that if the USB PHY is not powered and the corresponding USB controller nodes in the kernel DTS are not disabled, the kernel initialization will hang at the initialization of the USB controller. The typical log is as follows:

```
[ 1.240101] ohci_hcd: USB 1.1 'Open' Host Controller (OHCI) Driver
[ 1.241896] ohci-platform: OHCI generic platform driver
[ 1.242698] ohci-platform fd8c0000.usb: Generic Platform OHCI controller
[ 1.243686] ohci-platform fd8c0000.usb: new USB bus registered, assigned bus number 3
```

2.1.3 RK356x USB 3.0 PHYs Power Supply and Power Consumption Control

RK3566 USB 3.0 Host_1 utilizes the USB3/SATA/QSGMII Combo PHY_1 (corresponding to MULTI_PHY1 in the schematic diagram);

RK3568 USB 3.0 OTG uses the USB3/SATA Combo PHY_0 (corresponding to MULTI_PHY0 in the schematic diagram), and USB 3.0 Host_1 utilizes the USB3/SATA/QSGMII Combo PHY_1 (corresponding to MULTI_PHY1 in the schematic diagram);

The RK356x MULTI_PHY0/MULTI_PHY1/MULTI_PHY2 use the same power supply, as shown in Figure 2-5, with two power supplies: 0.9V and 1.8V. For the voltage requirements of these two power supplies, please refer to Table 4.

Table 4 USB 3.0 Combo PHY Power Supplies

Supply Voltage	Min	Typ	Max	Unit
MULTI_PHY_AVDD_1V8	1.62	1.8	1.98	V
MULTI_PHY_AVDD_0V9	0.81	0.9	0.99	V

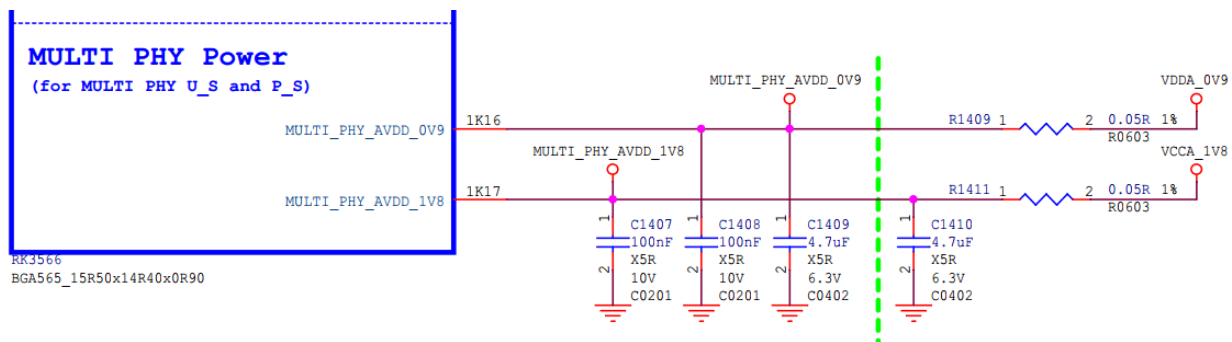


Figure 2-5 RK356x Multi PHYs Power Circuit

When the chip is powered on, the three MULTI_PHY0/MULTI_PHY1/MULTI_PHY2 are in the working state by default. In UBoot, the three PHYs will be set to the reset state through software settings to maintain the PHY at the lowest power consumption. After entering the kernel, the USB 3.0/SATA/PCIE/QSGMII controller driver will call the rockchip_combphy_init() function to release the PHY's reset.

When the Combo PHY operates in USB 3.0 mode, the PHY's PIPE state (P0/P1/P2/P3) is automatically controlled by the USB 3.0 controller hardware, dynamically entering and exiting P0/P1/P2/P3 states according to different working scenarios. For example, when no USB device is inserted into the USB 3.0 Host, the PIPE is in the P3 state; when a U3 disk is inserted, it switches to the P0 state; when a USB 3.0 HUB is connected, as long as the downstream port of the HUB is not connected to any other USB peripheral, the PIPE state will automatically enter the P3 state. When a USB peripheral is inserted into the USB3 HUB, the PIPE state switches to P0. (Note: P0 is the normal working state, and P3 is the lowest power consumption state)

When the Combo PHY is not used for any function, it is recommended to disable the corresponding USB 3.0 controller and PHY nodes in the kernel DTS to ensure the PHY is at the lowest power consumption state. For the configuration method of DTS, please refer to the section [RK356x USB DTS Configuration](#).

2.2 RK3566 USB 2.0 OTG Micro-B Interface Hardware Circuit

The RK3566 USB 2.0 OTG Micro-B interface hardware circuit is shown in Figures 2-6 to 2-8 and is similar to the common USB 2.0 OTG circuit design found in the SDKs of other Rockchip platforms. Therefore, the principles of the circuit design are not detailed here.

Three points need attention:

1. DP/DM must be connected in series with a $2.2\ \Omega$ resistor each to enhance the stress/electrostatic discharge resistance of DP/DM;
2. If USB Device dynamic plug detection and charging type detection functions are required, the VBUSDET pin must be connected to the USB Micro interface;
3. If USB Device dynamic plug detection and charging type detection functions are not required, it is permissible to fix the VBUSDET pin to 3.3V or leave it floating (it is recommended to pull up to 3.3V). Since the connection of RK356x Maskrom USB does not rely on the detection of the VBUSDET signal, even if the VBUSDET pin is left floating, Maskrom USB can still work normally and download firmware;

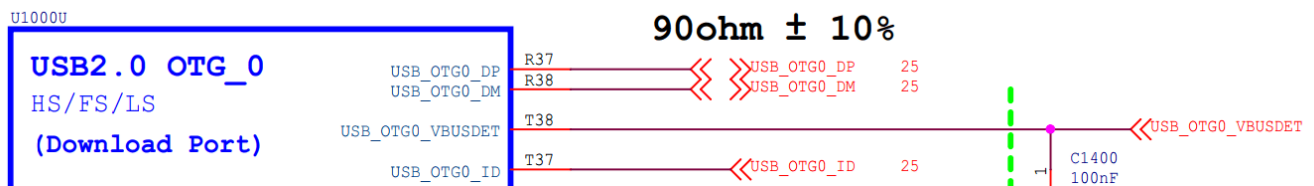


Figure 2-6 RK3566 USB2.0 OTG Pins

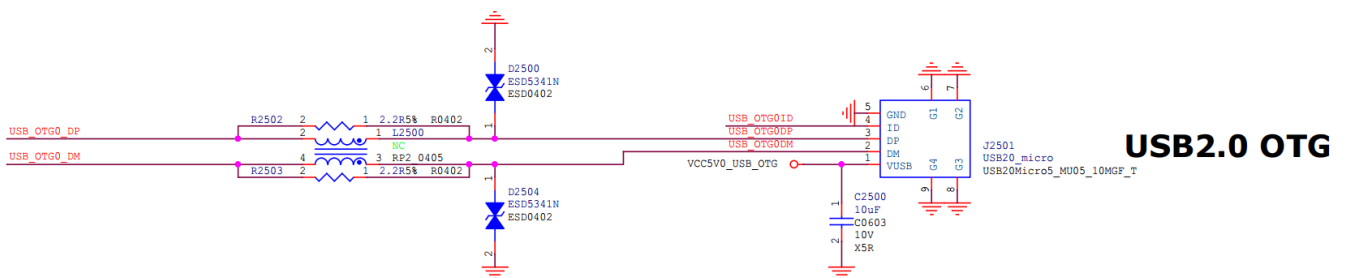


Figure 2-7 RK3566 USB2.0 OTG Micro-B Interface

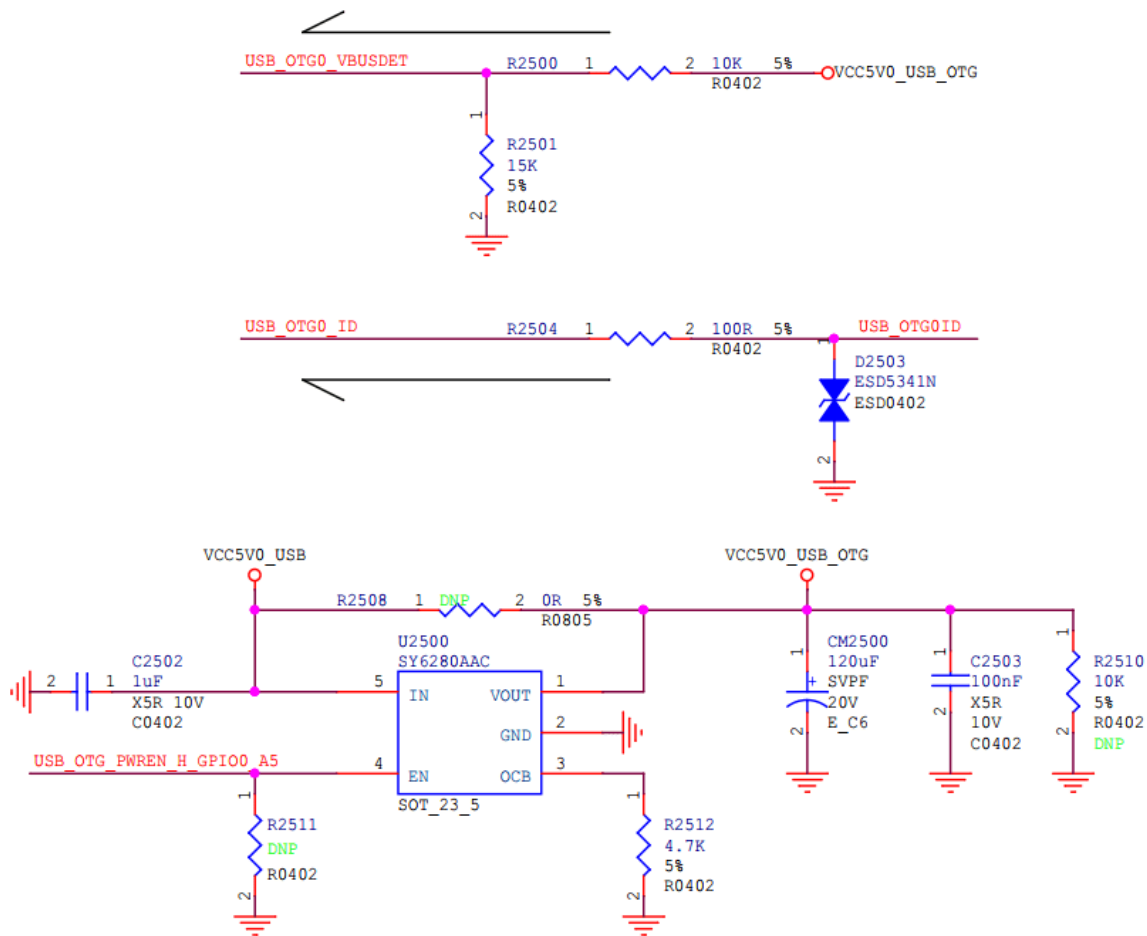


Figure 2-8 RK3566 USB2.0 OTG VBUS and OTG_ID Control Circuit

2.3 RK3566 USB 2.0 OTG Type-C Interface Hardware Circuitry

2.3.1 RK3566 Type-C USB 2.0 Only Hardware Circuit Design[No PD]

The hardware circuit design for RK3566 Type-C USB2 only is shown in Figures 2-9 to 2-11. The features of this circuit design are:

1. Support for USB 2.0 OTG mode detection and automatic switching;
2. No support for Type-C PD (Power Delivery);
3. No need for an external Type-C controller chip, saving hardware costs;

In this circuit design, the Type-C interface's DP1/DM1 and DP2/DM2 are directly connected to the RK3566 chip's USB_OTG0_DP/USB_OTG0_DM, eliminating the need to detect the orientation of the Type-C plug. The key point in the circuit design is to detect UFP/DFP types (corresponding to USB Device/Host mode) through CC1/CC2, as referenced in Figure 2-11. The design principle of the CC detection circuit is based on the USB Type-C Specification^[1] document's design specifications for 3.5.2 USB Type-C to USB 2.0 Standard-A Cable Assembly and 3.6.1 USB Type-C to USB 3.1 Standard-A Receptacle Adapter Assembly:

1. Type-C to USB 2.0 Standard-A Cable, Pin A5 (CC) of the USB Type-C plug shall be connected to VBUS through a resistor R_p ($56\text{ k}\Omega \pm 5\%$);
2. Type-C to USB 3.1 Standard-A Receptacle, Pin A5 (CC) of the USB Type-C plug shall be connected to GND

through a resistor R_d ($5.1\text{ k}\Omega \pm 20\%$);

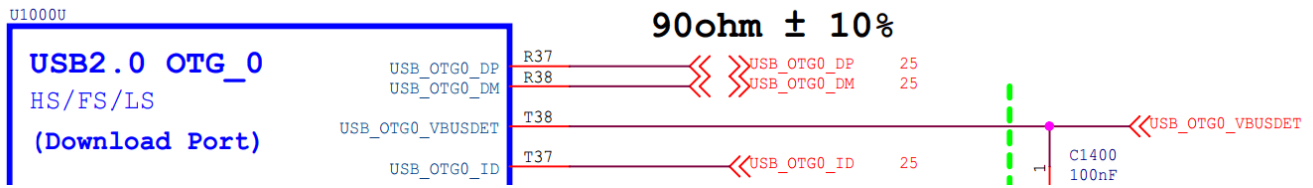


Figure 2-9 RK3566 USB2.0 OTG Pins

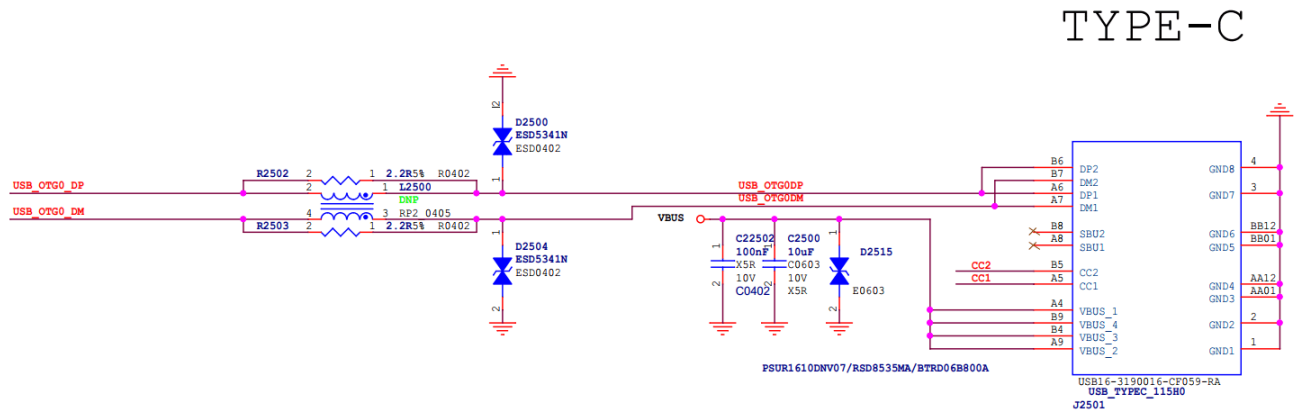


Figure 2-10 RK3566 USB 2.0 OTG Type-C Interface

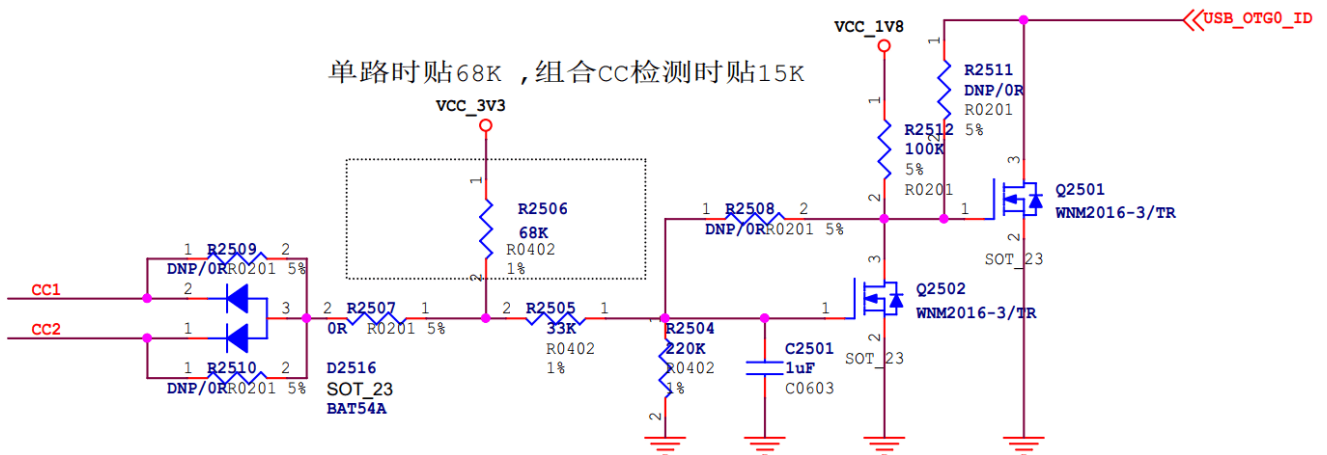


Figure 2-11 RK3566 USB 2.0 OTG Type-C CC Detection Circuit

Note:

1. The internal pull-up voltage of the RK3566 chip's USB_OTG0_ID is 1.8V, with an internal pull-up resistor of 155K ohm;
2. When a Type-C to USB 2.0 Standard-A Plug cable is inserted, the cable's CC line is weakly pulled up to VBUS 5V, ensuring that USB_OTG0_ID is at a high level, thus USB 2.0 OTG operates in Device mode;
3. When a Type-C to USB 2.0 Standard-A Receptacle cable is inserted, the cable's CC line is weakly pulled down to GND, and USB_OTG0_ID is also pulled to a low level, thus USB 2.0 OTG switches to Host mode based on the ID interrupt.

2.3.2 RK3566 Type-C USB 2.0 + PD Hardware Circuit

The hardware circuit design for RK3566 Type-C USB2 + PD is shown in Figures 2-12 to 2-14. The characteristics of this circuit design include:

1. Support for USB 2.0 OTG mode detection and automatic switching;
2. Support for Type-C PD (Power Delivery);
3. Requirement for an external controller chip for Type-C;
4. Requirement for a Type-C VBUS 5V voltage regulation circuit (to support PD high-voltage high-current charging);

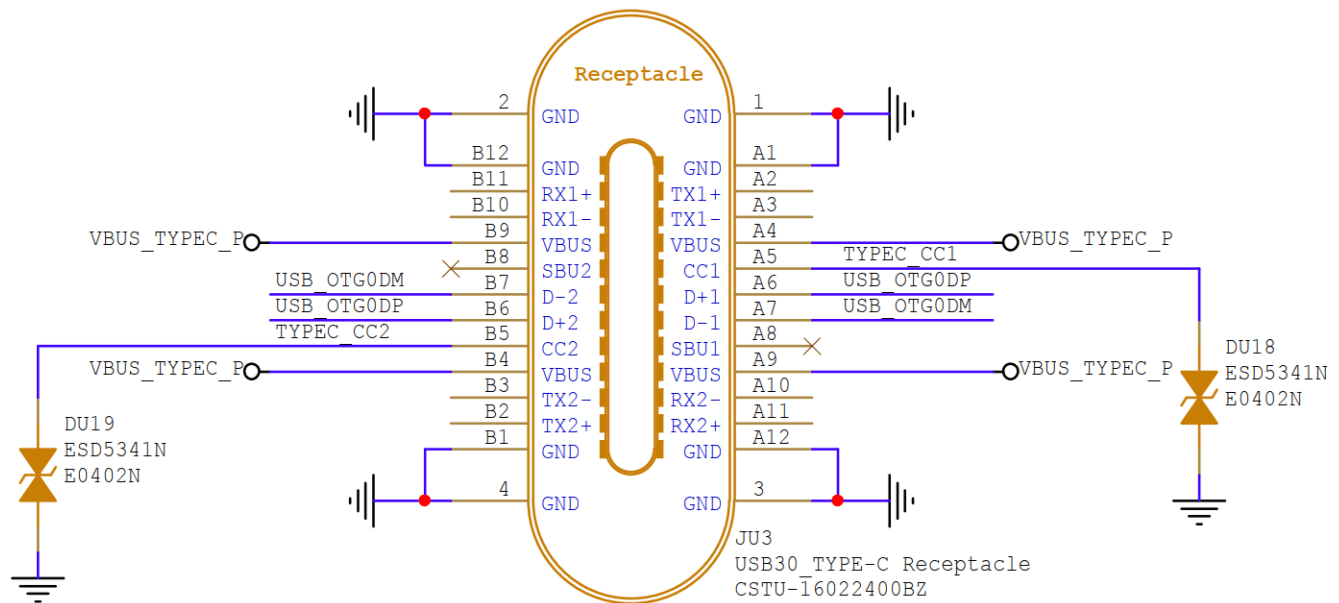


Figure 2-12 RK3566 Type-C USB 2.0 + PD Interface

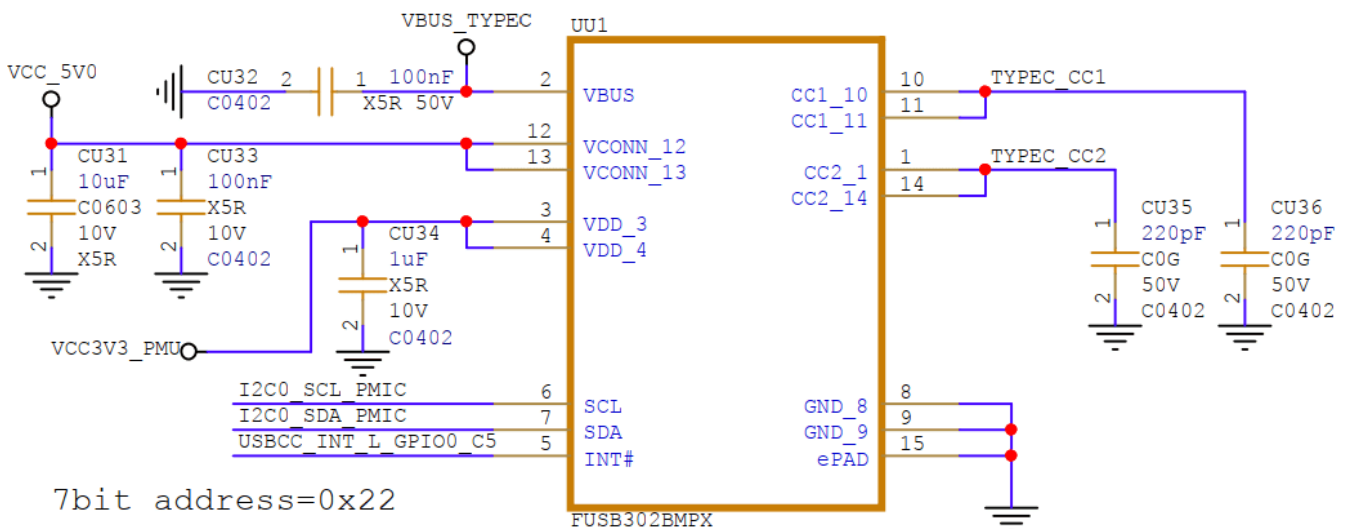


Figure 2-13 RK3566 Type-C FUSB302 Circuit

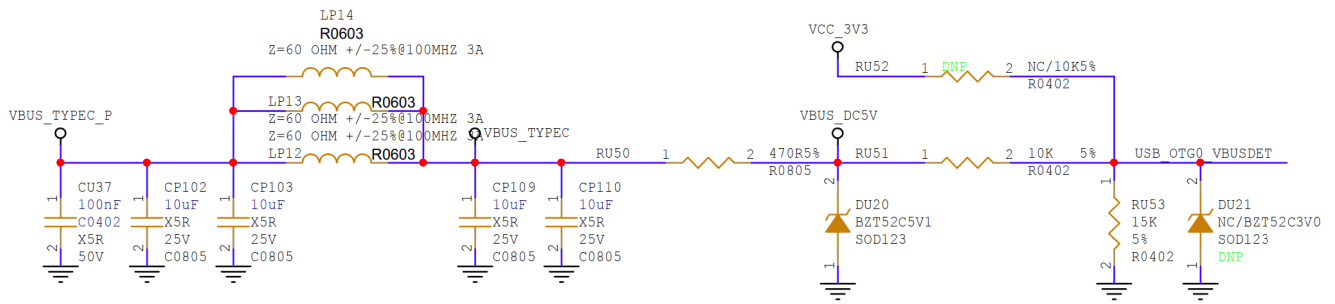


Figure 2-14 RK3566 Type-C VBUS 5V Voltage Regulation Circuit

2.4 RK356x USB 2.0 Host Type-A Interface Hardware Circuit

The RK356x USB 2.0 Host Type-A interface hardware circuit is shown in Figures 2-15 and 2-16, similar to the common USB 2.0 Host circuit design found in the SDKs of other Rockchip platforms. Therefore, the principles of the circuit design are not detailed here.

It should be noted that if the USB 2.0 Host is not used in the product, it is recommended to disconnect the power supply of the USB 2.0 Host PHY. For specific methods, refer to the section [RK356x USB 2.0 PHYs Power Supply and Power Consumption Control](#) and the circuit design in Figure 5.

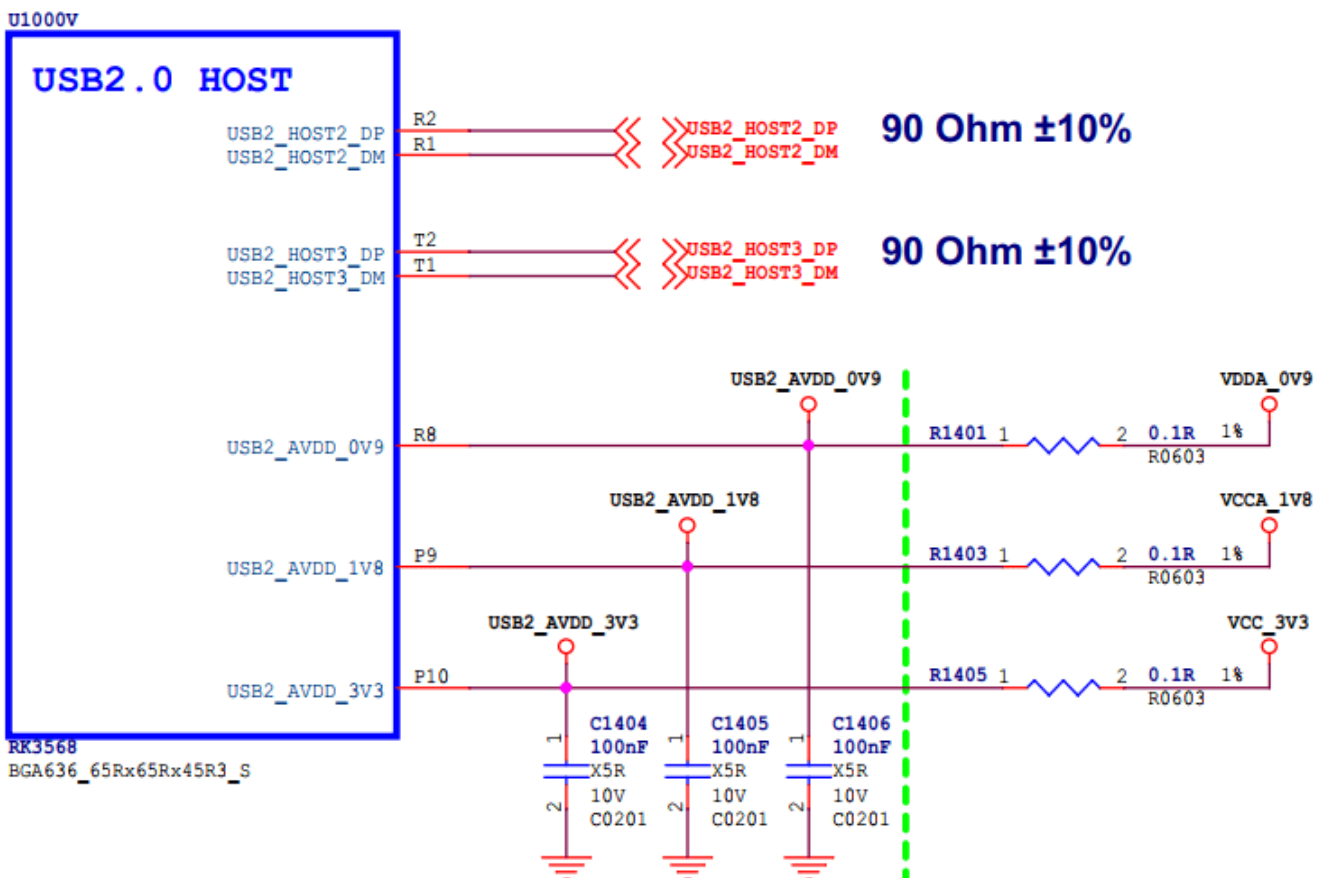
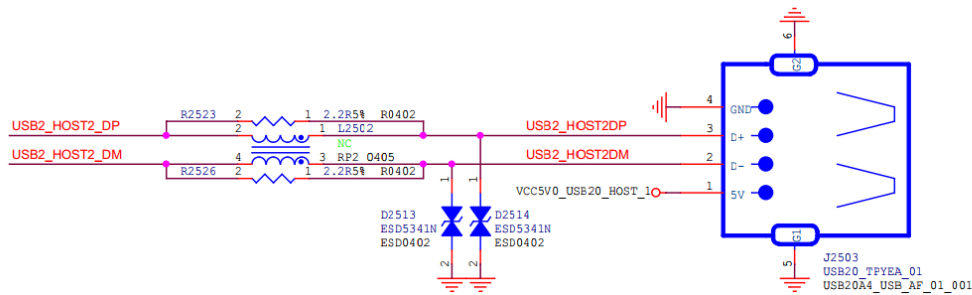


Figure 2-15 RK356x USB 2.0 Host pins



USB2.0 HOST2

Figure 2-16 RK356x USB 2.0 Host Interface

2.5 RK3568 USB 3.0 OTG Type-A Interface Hardware Circuit

The interface circuit design for RK3568 USB 3.0 OTG Type-A is shown in Figures 2-17 to 2-19. The advantage of this circuit design is that the physical interface is smaller in size compared to Micro USB 3.0 and is convenient for connecting USB peripherals. However, the disadvantage is that it does not support detection of USB3_OTG0_ID, hence it cannot support hardware automatic switching to OTG Host mode. Mode switching must be done through software, with the switch command referenced in the section [RK356x USB OTG Mode Switching Commands](#). Under normal circumstances, USB 3.0 OTG is configured by default in OTG mode, and when connected to a PC as Device mode, the USB driver supports automatic switching to Device mode. If a USB peripheral (such as a USB flash drive) needs to be connected, the mode must be forcibly switched to Host mode via software. When switching back from Host mode to Device mode, a software force switch is also required.

It should be noted that some USB 3.0 peripherals (such as USB 3.0 mechanical hard drives and USB 3.0 Cameras) have high current requirements. Therefore, when USB 3.0 OTG is operating in Host mode, it is necessary to ensure that the VBUS supply current reaches over 1A, as shown in Figure 2-19. The power chip SY6280AAC is configured to limit the output current to 1.446 A, meeting the current requirements for most USB 3.0 peripherals.

There are four points to pay attention to:

1. DP/DM must each be connected in series with a 2.2 Ω resistor to enhance the stress/electrostatic protection capabilities of DP/DM;
2. If dynamic plug detection and charging type detection functions for USB Device are to be supported, the VBUSDET pin must be connected to the USB Type-A interface;
3. If dynamic plug detection and charging type detection functions for USB Device are not required, it is permissible to keep the VBUSDET pin fixed at 3.3V or floating (it is recommended to pull up to 3.3V). Since the connection of RK356x Maskrom USB does not rely on the detection of the VBUSDET signal, even if the VBUSDET pin is floating, Maskrom USB can still operate normally and download firmware;
4. USB3_OTG0_ID is not used, so it can be left floating, with the chip internally pulled up to 1.8V;

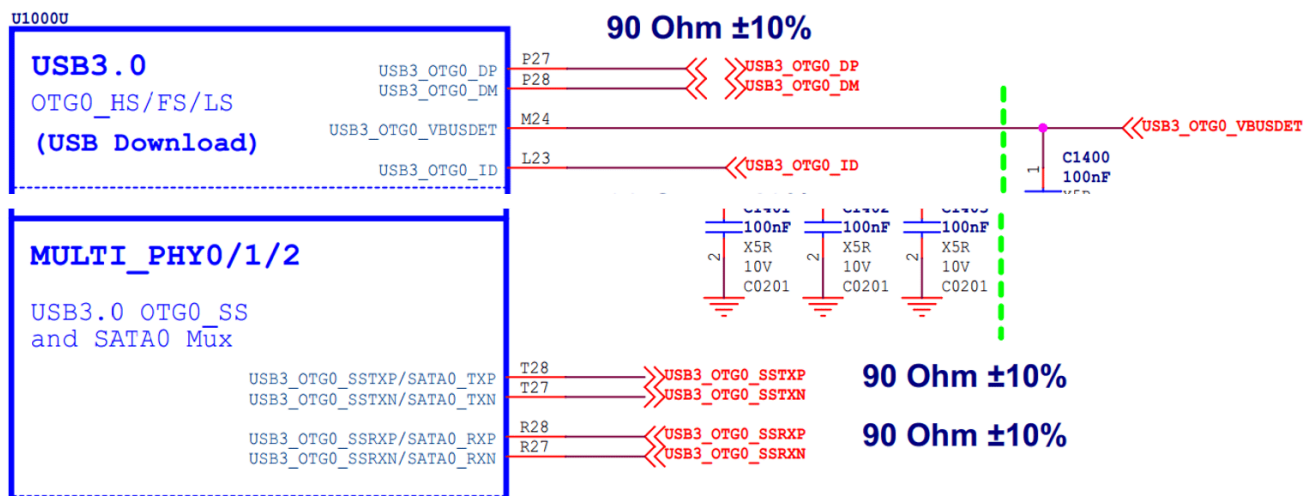


Figure 2-17 RK3568 USB 3.0 OTG Pins

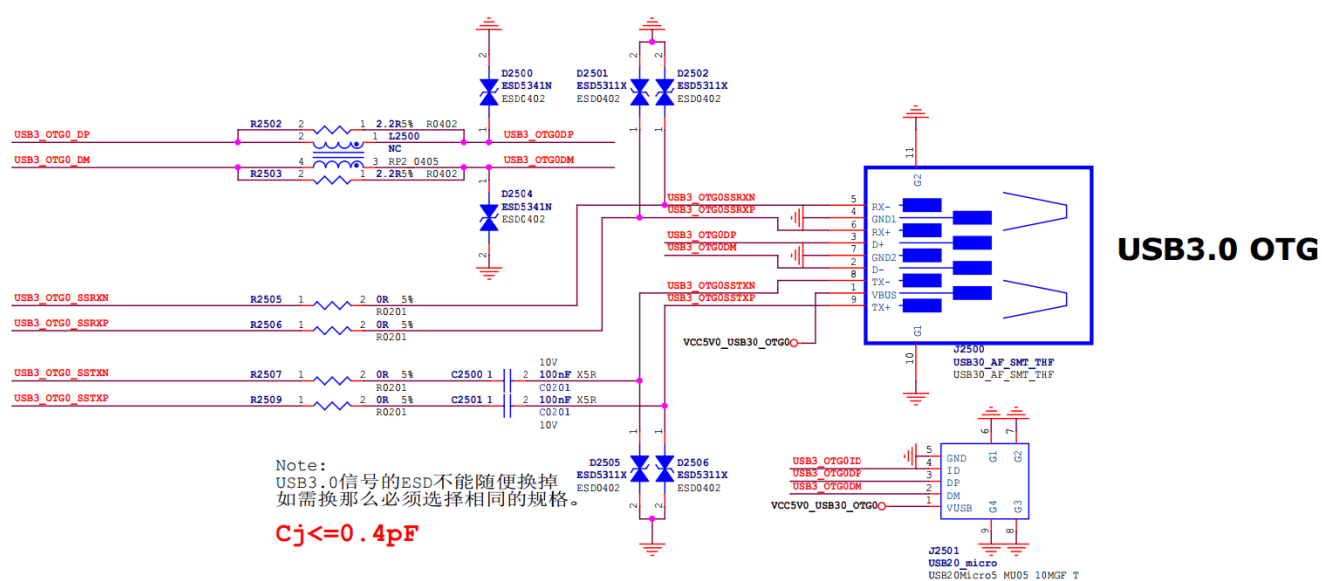


Figure 2-18 RK3568 USB 3.0 OTG Type-A Interface

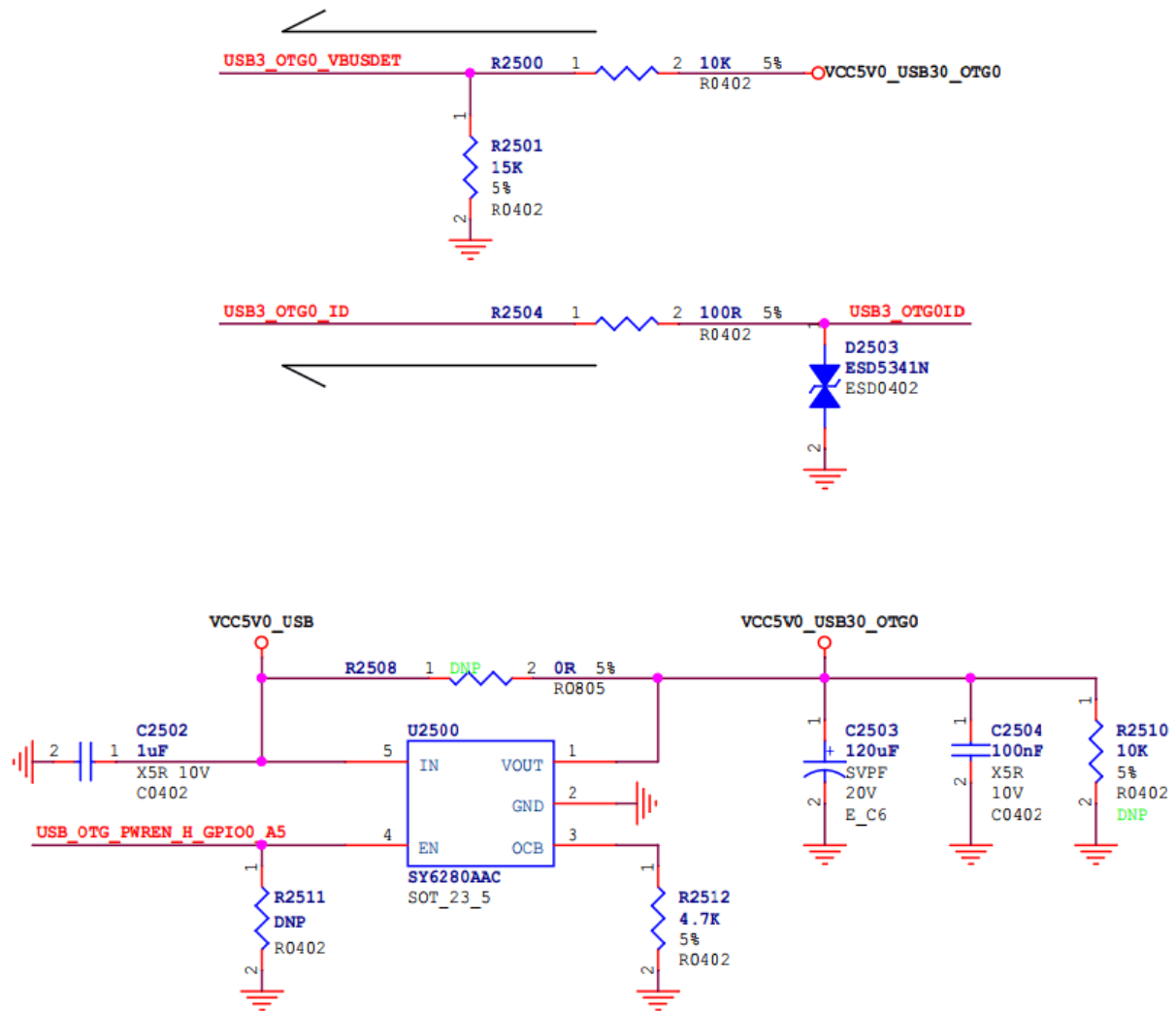


Figure 2-19 RK3568 USB 3.0 OTG VBUS and OTG ID Control Circuit

2.6 RK3568 USB 3.0 OTG Type-C Interface Hardware Circuitry

The RK3568 SDK does not provide a hardware reference circuit for the USB 3.0 OTG Type-C interface. If developers wish to support this interface, they need to add a USB 3.1 Switch chip (such as FUSB340) and a CC communication chip (such as FUSB302) between the RK3568 USB 3.0 OTG and Type-C interfaces.

2.7 RK356x USB 3.0 HOST Type-A Interface Hardware Circuit

The hardware circuit of the RK3566/RK3568 USB 3.0 HOST Type-A interface is essentially the same as the [RK3568 USB 3.0 OTG Type-A Interface Hardware Circuit](#) described in the previous sections, with the difference being that the RK3566/RK3568 USB 3.0 HOST lacks the VBUSDET pin and ID pin. For design considerations, please refer to the RK3568 USB 3.0 OTG Type-A Interface Hardware Circuit.

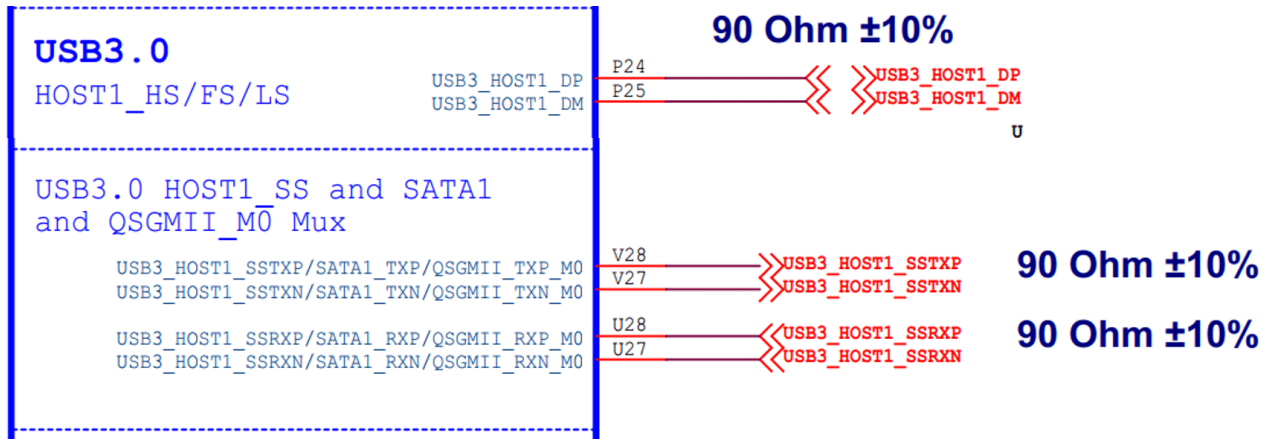


Figure 2-20 RK356x USB3.0 HOST1 Pins

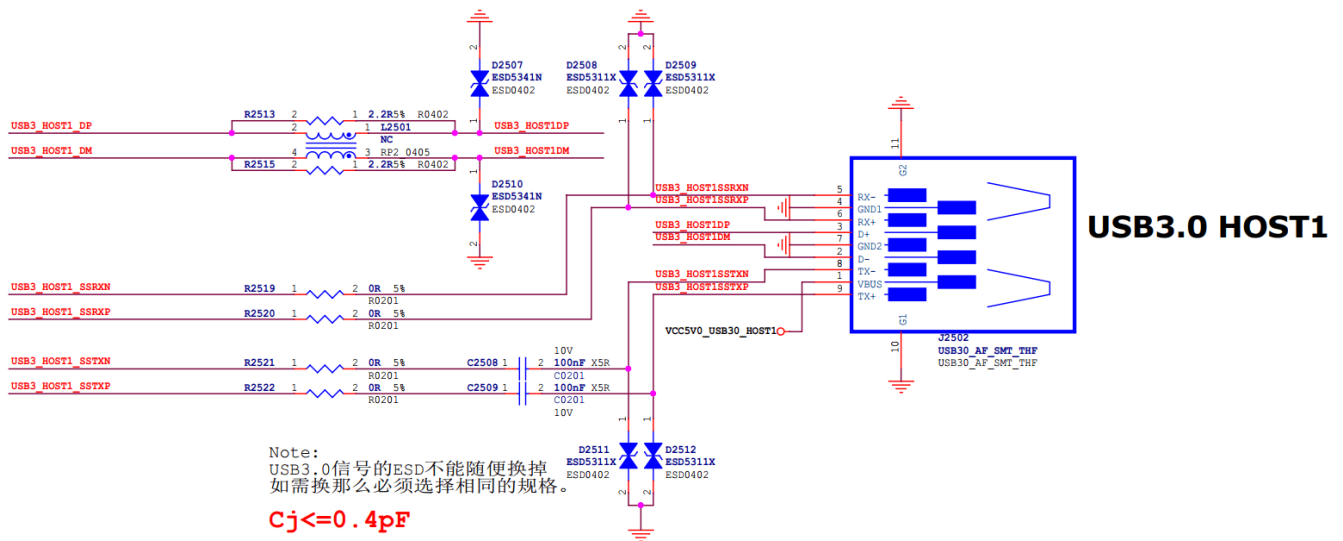


Figure 2-21 RK356x USB3.0 HOST1 Type-A Interface

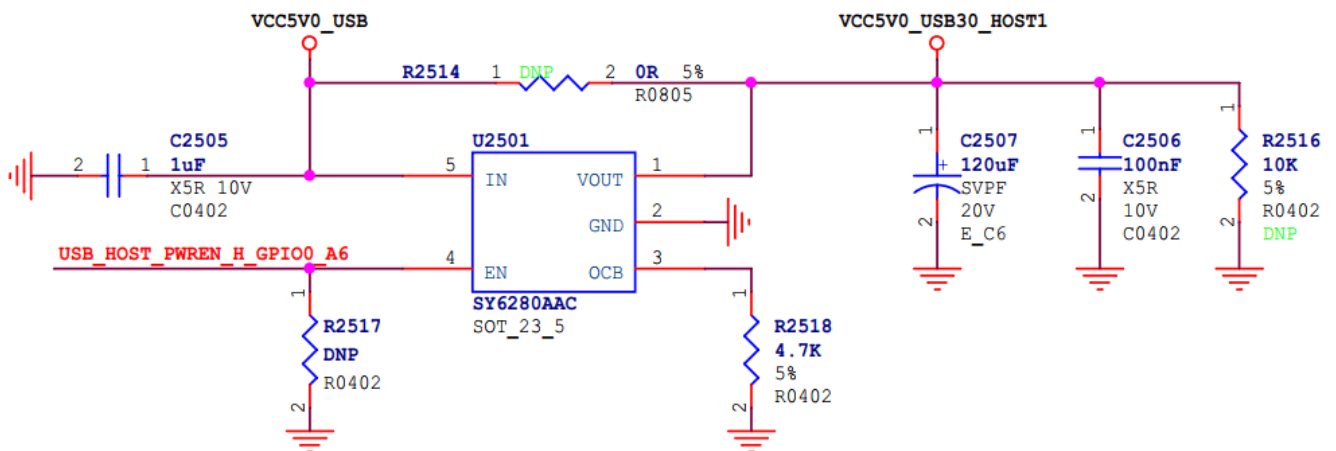


Figure 2-22 RK356x USB3.0 HOST1 VBUS Control Circuit

3. RK356x USB DTS Configuration

The RK356x USB hardware circuitry is characterized by its diversity, particularly the flexible and variable USB OTG port hardware circuitry, as well as the complex USB 3.0/SATA/QSGMII Combo PHY multiplexing relationships. Therefore, it is recommended that developers understand the hardware circuit design to correctly configure the USB-related controllers and PHY DTS nodes.

This document mainly illustrates the configuration method for USB DTS, without detailing the drivers for USB controllers and PHYs. If developers need to understand the detailed design of USB controllers and PHYs, please refer to the USB Developer Guide in the SDK^[2].

3.1 RK3568 USB Controller and PHY DTSI Nodes

For detailed configuration methods of the RK3568 USB controller and PHY's DTS, please refer to the kernel documentation[4]~[10].

The following code is the main node related to the USB controller and PHY in the DTSI.

```
/* USB 3.0 OTG/SATA Combo PHY_0 */
combphy0_us: phy@fe820000 {
    compatible = "rockchip,rk3568-naneng-combphy";
    reg = <0x0 0xfe820000 0x0 0x100>;
    #phy-cells = <1>;
    clocks = <&pmucru CLK_PCIEPHY0_REF>, <&cru PCLK_PIPEPHY0>,
            <&cru PCLK_PIPE>;
    clock-names = "refclk", "apbclk", "pipe_clk";
    assigned-clocks = <&pmucru CLK_PCIEPHY0_REF>;
    assigned-clock-rates = <24000000>;
    resets = <&cru SRST_P_PIPEPHY0>, <&cru SRST_PIPEPHY0>;
    reset-names = "combphy-apb", "combphy";
    rockchip,pipe-grf = <&pipegrf>;
    rockchip,pipe-phy-grf = <&pipe_phy_grf0>;
    status = "disabled";
};

/* USB 3.0 Host/SATA/QSGMII Combo PHY_1 */
combphy1_usq: phy@fe830000 {
    compatible = "rockchip,rk3568-naneng-combphy";
    reg = <0x0 0xfe830000 0x0 0x100>;
    #phy-cells = <1>;
    clocks = <&pmucru CLK_PCIEPHY1_REF>, <&cru PCLK_PIPEPHY1>,
            <&cru PCLK_PIPE>;
    clock-names = "refclk", "apbclk", "pipe_clk";
    assigned-clocks = <&pmucru CLK_PCIEPHY1_REF>;
    assigned-clock-rates = <24000000>;
    resets = <&cru SRST_P_PIPEPHY1>, <&cru SRST_PIPEPHY1>;
    reset-names = "combphy-apb", "combphy";
    rockchip,pipe-grf = <&pipegrf>;
    rockchip,pipe-phy-grf = <&pipe_phy_grf1>;
    status = "disabled";
};
```

```

};
/* USB OTG/USB Host_1 USB 2.0 Comb PHY_0 */
usb2phy0: usb2-phy@fe8a0000 {
    compatible = "rockchip,rk3568-usb2phy";
    reg = <0x0 0xfe8a0000 0x0 0x10000>;
    interrupts = <GIC_SPI 135 IRQ_TYPE_LEVEL_HIGH>;
    clocks = <&pmucru CLK_USBPHY0_REF>;
    clock-names = "phyclk";
    #clock-cells = <0>;
    assigned-clocks = <&cru USB480M>;
    assigned-clock-parents = <&usb2phy0>;
    clock-output-names = "usb480m_phy";
    rockchip,usbgrf = <&usb2phy0_grf>;
    status = "disabled";

    u2phy0_host: host-port {
        #phy-cells = <0>;
        status = "disabled";
    };

    u2phy0_otg: otg-port {
        #phy-cells = <0>;
        status = "disabled";
    };
};
/* USB Host_1/USB Host_2 USB 2.0 Comb PHY_0 */
usb2phy1: usb2-phy@fe8b0000 {
    compatible = "rockchip,rk3568-usb2phy";
    reg = <0x0 0xfe8b0000 0x0 0x10000>;
    interrupts = <GIC_SPI 136 IRQ_TYPE_LEVEL_HIGH>;
    clocks = <&pmucru CLK_USBPHY1_REF>;
    clock-names = "phyclk";
    #clock-cells = <0>;
    rockchip,usbgrf = <&usb2phy1_grf>;
    status = "disabled";

    u2phy1_host: host-port {
        #phy-cells = <0>;
        status = "disabled";
    };

    u2phy1_otg: otg-port {
        #phy-cells = <0>;
        status = "disabled";
    };
};
/* USB 3.0 OTG controller */
usbdrd30: usbdrd {
    compatible = "rockchip,rk3568-dwc3", "rockchip,rk3399-dwc3";
    clocks = <&cru CLK_USB30TG0_REF>, <&cru CLK_USB30TG0_SUSPEND>,
            <&cru ACLK_USB30TG0>, <&cru PCLK_PIPE>;
    clock-names = "ref_clk", "suspend_clk",
                  "bus_clk", "pipe_clk";
};

```

```

#address-cells = <2>;
#size-cells = <2>;
ranges;
status = "disabled";

usbdrd_dwc3: dwc3@fcc00000 {
    compatible = "snps,dwc3";
    reg = <0x0 0xfcc00000 0x0 0x400000>;
    interrupts = <GIC_SPI 169 IRQ_TYPE_LEVEL_HIGH>;
    dr_mode = "otg";
    phys = <&u2phy0_otg>, <&combphy0_us PHY_TYPE_USB3>;
    phy-names = "usb2-phy", "usb3-phy";
    phy_type = "utmi_wide";
    power-domains = <&power RK3568_PD_PIPE>;
    resets = <&cru SRST_USB30TG0>;
    reset-names = "usb3-otg";
    snps,dis_enblslpm_quirk;
    snps,dis-u1-entry-quirk;
    snps,dis-u2-entry-quirk;
    snps,dis-u2-freeclk-exists-quirk;
    snps,dis-del-phy-power-chg-quirk;
    snps,dis-tx-ipgap-linecheck-quirk;
    snps,dis_rxdet_inp3_quirk;
    snps,xhci-trb-ent-quirk;
    status = "disabled";
};

};
/* USB 3.0 Host_1 controller */
usbhost30: usbhost {
    compatible = "rockchip,rk3568-dwc3", "rockchip,rk3399-dwc3";
    clocks = <&cru CLK_USB30TG1_REF>, <&cru CLK_USB30TG1_SUSPEND>,
            <&cru ACLK_USB30TG1>, <&cru PCLK_PIPE>;
    clock-names = "ref_clk", "suspend_clk",
                  "bus_clk", "pipe_clk";
    #address-cells = <2>;
    #size-cells = <2>;
    ranges;
    status = "disabled";

    usbhost_dwc3: dwc3@fd000000 {
        compatible = "snps,dwc3";
        reg = <0x0 0xfd000000 0x0 0x400000>;
        interrupts = <GIC_SPI 170 IRQ_TYPE_LEVEL_HIGH>;
        dr_mode = "host";
        phys = <&u2phy0_host>, <&combphy1_usq PHY_TYPE_USB3>;
        phy-names = "usb2-phy", "usb3-phy";
        phy_type = "utmi_wide";
        power-domains = <&power RK3568_PD_PIPE>;
        resets = <&cru SRST_USB30TG1>;
        reset-names = "usb3-host";
        snps,dis_enblslpm_quirk;
        snps,dis-u2-freeclk-exists-quirk;
        snps,dis-del-phy-power-chg-quirk;

```

```

        snps,dis-tx-ipgap-linecheck-quirk;
        snps,dis_rxdet_inp3_quirk;
        snps,xhci-trb-ent-quirk;
        status = "disabled";
    };
};

/* USB 2.0 Host_2 EHCI controller for high speed */
usb_host0_ehci: usb@fd800000 {
    compatible = "generic-ehci";
    reg = <0x0 0xfd800000 0x0 0x40000>;
    interrupts = <GIC_SPI 130 IRQ_TYPE_LEVEL_HIGH>;
    clocks = <&cru HCLK_USB2HOST0>, <&cru HCLK_USB2HOST0_ARB>,
            <&cru PCLK_USB>, <&usb2phy1>;
    clock-names = "usbhost", "arbiter", "pclk", "utmi";
    phys = <&u2phy1_otg>;
    phy-names = "usb2-phy";
    status = "disabled";
};

/* USB 2.0 Host_2 OHCI controller for full/low speed */
usb_host0_ohci: usb@fd840000 {
    compatible = "generic-ohci";
    reg = <0x0 0xfd840000 0x0 0x40000>;
    interrupts = <GIC_SPI 131 IRQ_TYPE_LEVEL_HIGH>;
    clocks = <&cru HCLK_USB2HOST0>, <&cru HCLK_USB2HOST0_ARB>,
            <&cru PCLK_USB>, <&usb2phy1>;
    clock-names = "usbhost", "arbiter", "pclk", "utmi";
    phys = <&u2phy1_otg>;
    phy-names = "usb2-phy";
    status = "disabled";
};

/* USB 2.0 Host_3 EHCI controller for high speed */
usb_host1_ehci: usb@fd880000 {
    compatible = "generic-ehci";
    reg = <0x0 0xfd880000 0x0 0x40000>;
    interrupts = <GIC_SPI 133 IRQ_TYPE_LEVEL_HIGH>;
    clocks = <&cru HCLK_USB2HOST1>, <&cru HCLK_USB2HOST1_ARB>,
            <&cru PCLK_USB>, <&usb2phy1>;
    clock-names = "usbhost", "arbiter", "pclk", "utmi";
    phys = <&u2phy1_host>;
    phy-names = "usb2-phy";
    status = "disabled";
};

/* USB 2.0 Host_3 OHCI controller for full/low speed */
usb_host1_ohci: usb@fd8c0000 {
    compatible = "generic-ohci";
    reg = <0x0 0xfd8c0000 0x0 0x40000>;
    interrupts = <GIC_SPI 134 IRQ_TYPE_LEVEL_HIGH>;
    clocks = <&cru HCLK_USB2HOST1>, <&cru HCLK_USB2HOST1_ARB>,
            <&cru PCLK_USB>, <&usb2phy1>;
    clock-names = "usbhost", "arbiter", "pclk", "utmi";
    phys = <&u2phy1_host>;
    phy-names = "usb2-phy";
    status = "disabled";
};

```



```
};
```

3.2 RK3568 USB 3.0 OTG to USB 2.0 Only Configuration

The USB 3.0 PHY of the USB 3.0 OTG shares the Comb PHY_0 with the SATA_0 controller. When Comb PHY_0 is configured for SATA_0, USB 3.0 OTG needs to operate in USB 2.0 only mode. The corresponding DTS configuration is as follows (no changes are required for the controller and PHY drivers):

```
&usbdrd_dwc3 {
    phys = <&u2phy0_otg>; /* Configure the phys property to reference only the USB 2.0
    phy node */
    phy-names = "usb2-phy";
    extcon = <&usb2phy0>;
    maximum-speed = "high-speed"; /* Configure the dwc3 controller to support up to high
    speed */
    snps,dis_u2_susphy_quirk; /* Configure the dwc3 controller to not support automatic
    suspend of usb2 phy */
    snps,usb2-lpm-disable; /* Configure the dwc3 host mode to not support LPM
    functionality */
};

&combphy0_us {
    rockchip,dis-u3otg0-port; /* Configure the dwc3_0 controller to support up to high
    speed */
    status = "okay";
};

&u2phy0_otg {
    rockchip,dis-u2-susphy; /* Optional, used to disable the PHY's dynamic entry into
    suspend mode, improving USB hot-plug stability */
    status = "okay";
};
```

3.3 RK356x USB 3.0 Host to USB 2.0 only Configuration

The USB 3.0 PHY of the USB 3.0 Host shares the Comb PHY_1 with the SATA_1/QSGMII controller. When Comb PHY_1 is configured for SATA_1 or QSGMII, the USB 3.0 Host needs to operate in USB 2.0 only mode. The corresponding DTS configuration is as follows (no changes are needed for the controller and PHY drivers):

```

&usbhost_dwc3 {
    phys = <&u2phy0_host>; /* Configure the phys property to reference only the USB 2.0
PHY node */
    phy-names = "usb2-phy";
    maximum-speed = "high-speed"; /* Configure the dwc3 controller to support up to high
speed */
    snps,dis_u2_susphy_quirk; /* Configure the dwc3 controller to not support automatic
suspend of USB2 PHY */
    snps,usb2-lpm-disable; /* Configure the dwc3 host mode to not support LPM
functionality */
    status = "okay";
};

&combphy1_usq {
    rockchip,dis-u3otg1-port; /* Configure the dwc3_1 controller to support up to high
speed */
    status = "okay";
};

```

3.4 RK356x Type-C USB DTS Configuration

Common RK356x Type-C design schemes include the following five types:

1. RK3566 Type-C USB 2.0 only;
2. RK3568 Type-C USB 2.0 only;
3. RK3566 Type-C USB 2.0 + PD;
4. RK3568 Type-C USB 2.0 + PD;
5. RK3568 Type-C USB 3.0 + PD;

For schemes 1 and 2, the hardware circuit can refer to the section [RK3566 Type-C USB 2.0 only Hardware Circuit Design\[No PD\]](#), which adopts a hardware conversion circuit from CC to OTG_ID, and the principle of detecting OTG mode internally by the chip is the same as the Micro-B USB interface. Therefore, the USB DTS configuration for schemes 1 and 2 can refer to the common RK356x OTG Micro-USB /Type-A interface design.

For schemes 3 and 4, the hardware circuit can refer to the section [RK3566 Type-C USB 2.0 + PD Hardware Circuit](#), which uses an external Type-C controller chip to support PD communication. The USB DTS configuration for these two schemes is essentially the same. This section focuses on explaining the USB DTS configuration method for scheme 3 (RK3566 Type-C USB 2.0 + PD). Scheme 4 only needs to refer to the section [RK3568 USB 3.0 OTG to USB 2.0 only Configuration](#) on this basis, to configure the USB controller to work in USB 2.0 only mode.

For scheme 5, considering that the RK3568 USB3 PHY does not support Type-C front and back switching, it is necessary to add two external chips in the hardware circuit, namely: a USB3 Switch chip (such as FUSB340) and a Type-C controller chip (such as FUSB302). The DTS configuration of the Type-C controller chip can refer to schemes 3 and 4, while the control of the USB 3.1 Switch chip needs to be developed based on the hardware scheme selection.

3.4.1 Linux-4.19 RK3566 Type-C USB 2.0 + FUSB302 DTS Configuration

The Linux-4.19 kernel supports two sets of FUSB302 drivers, namely:

`drivers/mfd/fusb302.c` [Legacy]

`drivers/usb/typec/tcpm/fusb302.c` [New]

The Linux-4.19 SDK defaults to using the Legacy driver, and `rockchip_defconfig` has enabled the corresponding driver configuration `CONFIG_FUSB_30X`. This driver has been promoted and mass-produced on the RK3399 platform, and it is recommended to use it preferentially. If the new driver is used, refer to [Linux-4.19 RK3566 Type-C USB 2.0 + HUSB311 DTS Configuration](#) to update the kernel Type-C TCPM series patches.

Taking the adaptation of the Legacy FUSB302 driver as an example, the reference configuration for Linux-4.19 DTS is as follows:

```
/* Modify according to the actual hardware design of FUSB302 I2C */
&i2cx {
    fusb0: fusb30x@22 { /* Configure hardware information of FUSB302 chip */
        compatible = "fairchild,fusb302";
        reg = <0x22>;
        pinctrl-names = "default";
        pinctrl-0 = <&fusb0_int>;
        int-n-gpios = <&gpiox RK_Pxx GPIO_ACTIVE_HIGH>; /* Modify according to the
actual connected GPIO of FUSB302 INT */
        vbus-5v-gpios = <&gpiox RK_Pxx GPIO_ACTIVE_HIGH>; /* Modify according to the
actual connected GPIO of Type-C VBUS */
        status = "okay";
    };
};

/* Enable USB2.0 PHY */
&usb2phy0 {
    status = "okay";
    extcon = <&fusb0>; /* Configure extcon property to receive Type-C DRD notifier from
FUSB302 driver */
};

/* Enable USB2.0 PHY OTG port */
&u2phy0_otg {
    rockchip,dis-u2-susphy; /* Optional, used to disable the dynamic suspend function of
PHY to improve USB hot-plug stability */
    status = "okay";
};

/* Enable parent node of USB3 controller */
&usbdrd30 {
    status = "okay";
};

/* Enable child node of USB3 controller */
&usbdrd_dwc3 {
```

```

        extcon = <&fusb0>; /* Configure extcon property to receive Type-C DRD notifier from
FUSB302 driver */
        status = "okay";
};

&pinctrl {
    fusb30x { /* Configure interrupt and vbus gpio pinctrl for FUSB302 driver */
        fusb0_int: fusb0-int {
            rockchip,pins = <gpiox RK_Pxx RK_FUNC_GPIO &pcfg_pull_up>,
                           <gpiox RK_Pxx RK_FUNC_GPIO &pcfg_pull_up>;
        };
    };
};
};

```

3.4.2 Linux-4.19 RK3566 Type-C USB 2.0 + HUSB311 DTS Configuration

The latest Linux-4.19 kernel for RK3568 already supports the Type-C controller chips HUSB311 and ET7303. Considering that both HUSB311 and ET7303 have the same I2C device address of 4E, and their DTS configurations are essentially the same, only the node names and the "compatible" attribute need to be modified. Therefore, this section uses HUSB311 as an example for explanation.

Both HUSB311 and ET7303 chips are based on the kernel Type-C driver TCPM framework, thus requiring Linux-4.19 to be synchronized with the latest code from develop-4.19. If it is not possible to synchronize the complete kernel code, at least the following directories should be synchronized:

```

kernel/include/linux/usb
kernel/drivers/usb/typec
kernel/drivers/usb/dwc3

```

Additionally, manually update the following patches:

```

0001-usb-typec-tcpm-add-vdm-retry-mechanism.patch
0002-arm64-rockchip_defconfig-enable-tcpm-and-husb311-and.patch

```

Please download the complete reference material: "RK356x_Linux-4.19_TypeC-HUSB311-ET7303-Reference Design" for modification.

Download link: <https://redmine.rockchip.com.cn/documents/113>

Taking the adaptation of the HUSB311 driver on RK3566 as an example, the Linux-4.19 DTS reference configuration is as follows:

```

/* Include necessary header files */
#include "dt-bindings/usb/pd.h"

/* Optional configuration for hardware solutions where vbus is controlled through gpio */
vcc_otg_vbus: otg-vbus-regulator {
    compatible = "regulator-fixed";
    gpio = <&gpiox RK_PXX GPIO_ACTIVE_HIGH>; /* Modify according to the actual GPIO
connected to Type-C VBUS */

```

```

pinctrl-names = "default";
pinctrl-0 = <&otg_vbus_drv>;
regulator-name = "vcc_otg_vbus";
regulator-min-microvolt = <5000000>;
regulator-max-microvolt = <5000000>;
enable-active-high;
};

/* Modify according to the actual hardware design of HUSB311 I2C */
&i2cx {
    /* For et7303, change to usbc0: et7303@4e */
    usbc0: husb311@4e {
        /* For et7303, compatible = "etek,et7303"*/
        compatible = "hynetek,husb311";
        reg = <0x4e>;
        /* Modify according to the actual GPIO connected to HUSB311 INT */
        interrupt-parent = <&gpiox>;
        interrupts = <X IRQ_TYPE_LEVEL_LOW>;
        pinctrl-names = "default";
        pinctrl-0 = <&husb311_int>;
        /*
         * vbus configuration
         * If vbus is controlled through gpio, then vbus-supply = <&vcc_otg_vbus>
configuration is needed;
         * If vbus is output by RK817/RK818/bq25700/other charge ic, then no vbus-
supply = <&vcc_otg_vbus> configuration is needed;
         */
        vbus-supply = <&vcc_otg_vbus>;
        status = "okay";

        usb_con: connector {
            data-role = "dual";
            power-role = "dual";
            try-power-role = "sink";
            /*
             * source-pdos: PDO_FIXED(voltage(mV), current(mA),
PDO_FIXED_USB_COMM)
             * When the motherboard acts as Source, the list of voltage/current
that the motherboard vbus can output; for RK platform, use default values
             */
            source-pdos = <PDO_FIXED(5000, 2000, PDO_FIXED_USB_COMM)>;
            /*
             * sink-pdos: <PDO_FIXED(voltage(mV), current(mA),
PDO_FIXED_USB_COMM)>
             * When the motherboard acts as Sink, the list of voltage/current
that the motherboard can support.
             * If there is a PD fast-charging requirement, sink-pdos
configuration depends on the maximum voltage/current of charger ic charging input and the
motherboard's endurance
             * If the voltage is too high, it may burn the motherboard;
communicate with the hardware engineer for this detail.

```

```

        * Below is the configuration for the maximum charging
voltage(20v)/current(3A) supported by the motherboard; modify according to hardware design
and actual requirements.
        * For example, if the motherboard only supports a maximum charging
voltage(12v)/current(3A), remove 15v/3A and 20V/3A.
        */
sink-pdos = <PDO_FIXED(5000, 3000, PDO_FIXED_USB_COMM)
            PDO_FIXED(9000, 3000, PDO_FIXED_USB_COMM)
            PDO_FIXED(12000, 3000, PDO_FIXED_USB_COMM)
            PDO_FIXED(15000, 3000, PDO_FIXED_USB_COMM)
            PDO_FIXED(20000, 3000, PDO_FIXED_USB_COMM)>;

        /* If there is no PD fast-charging requirement, the sink-pdos
configuration can be as follows. */
sink-pdos = <PDO_FIXED(5000, 2000, PDO_FIXED_USB_COMM)>;
        /* Charging power(op-sink-microwatt) = max_vol(mV) * max_cur(mA),
use default */
        op-sink-microwatt = <10000000>;

    };

};

&pinctrl {
    husb311 {
        husb311_int: husb311-int {
            rockchip,pins = <X RK_PXX RK_FUNC_GPIO &pcfg_pull_up>;
        };
    };

    usb {
        otg_vbus_drv: otg-vbus-drv {
            /* Optional, if there is no vcc_otg_vbus, this configuration is not
needed */
            rockchip,pins = <RK_XX RK_FUNC_GPIO &pcfg_pull_none>;
        };
    };
};

/* Enable PMIC */
bq25700: bq25700@6b {
    compatible = "ti,bq25703";
    reg = <0x6b>;
    extcon = <&usbc0>; /* Configure extcon property to receive TCPM driver's PD notifier
*/
    .....
};

/* Enable USB2.0 PHY */
&u2phy0 {
    status = "okay";
    extcon = <&usbc0>; /* Configure extcon property to receive TCPM driver's Type-C DRD
notifier */
};

```

```

/* Enable USB2.0 PHY OTG port */
&u2phy0_otg {
    rockchip,dis-u2-susphy; /* Optional, used to disable the PHY's dynamic suspend
function to improve USB hot-plug stability */
    status = "okay";
};

/* Enable USB3 controller parent node */
&usbdrd30 {
    status = "okay";
};

/* Enable USB3 controller child node */
&usbdrd_dwc3 {
    status = "okay";
    extcon = <&usbc0>; /* Configure extcon property to receive TCPM driver's Type-C DRD
notifier */
};

```

USB VBUS Controlled by RK817/RK818/bq25700 and Other PMICs DTS Configuration Instructions

```

/* rk818 */
rk818: pmic@1c {
    extcon = <&usbc0>;
};

/* rk817 */
rk817: pmic@20 {
    charger {
        extcon = <&usbc0>;
    };
};

/* bq25700 */
bq25700: bq25700@09 {
    extcon = <&usbc0>;
};

```

3.4.3 Linux-5.10 RK3566 Type-C USB 2.0 + FUSB302 DTS Configuration

The Linux-5.10 Type-C driver employs the standard TCPM framework, and the most significant difference from the Linux-4.19 and earlier kernel versions is that the Linux-5.10 Type-C driver uses the mechanism of remote-endpoint and registration of callback functions, replacing the previous extcon mechanism. Therefore, in the DTS configuration related to Linux-5.10 Type-C, the "extcon" property is no longer required.

Taking the RK3566 adaptation of the FUSB302 driver as an example, the reference configuration for Linux-5.10 DTS is as follows:

```

/* Configure VBUS gpio regulator for Type-C USB Host mode VBUS 5V output control */
vbus_typec: vbus-typec-regulator {

```

```

compatible = "regulator-fixed";
enable-active-high;
gpio = <&gpiox RK_PXX GPIO_ACTIVE_HIGH>;
pinctrl-names = "default";
pinctrl-0 = <&vcc5v0_typec0_en>;
regulator-name = "vbus_typec";
vin-supply = <&vcc5v0_sys>;
};

/* Configure FUSB302 chip hardware information */
&i2cX { /* Modify according to the actual hardware design of FUSB302 I2C */
    usbc0: fusb302@22 {
        compatible = "fcs,fusb302";
        reg = <0x22>;
        interrupt-parent = <&gpix>; /* Modify according to the actual GPIO
connection of FUSB302 INT */
        interrupts = <RK_PXX IRQ_TYPE_LEVEL_LOW>;
        pinctrl-names = "default";
        pinctrl-0 = <&usbc0_int>;
        vbus-supply = <&vbus_typec>;
        status = "okay";

        ports {
            #address-cells = <1>;
            #size-cells = <0>;
            /* Used to associate USB controllers to implement USB role switching
functionality */
            port@0 {
                reg = <0>;
                usbc0_role_sw: endpoint@0 {
                    remote-endpoint = <&dwc3_role_switch>;
                };
            };
        };

        usb_con: connector { /* Used to describe PD protocol-related information */
            compatible = "usb-c-connector";
            label = "USB-C";
            data-role = "dual";
            power-role = "dual";
            try-power-role = "sink";
            op-sink-microwatt = <1000000>;
            sink-pdos =
                <PDO_FIXED(5000, 2500, PDO_FIXED_USB_COMM)>;
            source-pdos =
                <PDO_FIXED(5000, 1500, PDO_FIXED_USB_COMM)>;
        };
    };
};

/* Enable Type-C0 USB2.0 PHY */
&u2phy0 {
    status = "okay";
};

```



```

};

&u2phy0_otg {
    rockchip,dis-u2-susphy; /* Optional, used to disable the PHY's dynamic suspend entry
feature, improving USB hot-plug stability */
    status = "okay";
};

/* Enable USB3 controller parent node */
&usbdrd30 {
    status = "okay";
};

/* Enable USB3 controller child node */
&usbdrd_dwc3 {
    status = "okay";
    usb-role-switch; /* Used to enable DWC3 driver to register USB Role Switch callback
functions */
    port {
        #address-cells = <1>;
        #size-cells = <0>;
        dwc3_role_switch: endpoint@0 {
            reg = <0>;
            remote-endpoint = <&usb0_role_sw>; /* Associate with FUSB302 to
implement USB role switching functionality */
        };
    };
};

&pinctrl {
    usb-typec {
        usb0_int: usb0-int { /* Configure FUSB302 driver's interrupt gpio pinctrl
*/
            rockchip,pins = <X RK_PXX RK_FUNC_GPIO &pcfg_pull_up>;
        };

        vcc5v0_typec0_en: vcc5v0-typec0-en { /* Configure FUSB302 driver's VBUS gpio
pinctrl */
            rockchip,pins =
                <X RK_PXX RK_FUNC_GPIO &pcfg_pull_none>;
        };
    };
};
};

```

3.4.4 Linux-5.10 RK3566 Type-C USB 2.0 + HUSB311 DTS Configuration

Replace the FUSB302 chip with the HUSB311 chip, and simply modify based on [Linux-5.10 RK3566 Type-C USB 2.0 + FUSB302 DTS Configuration](#). Refer to the modifications:

```

/* Configure the external Type-C controller chip HUSB311 */
&i2cx {
    usbc0: husb311@4e {
        compatible = "hynetek,husb311";
        reg = <0x4e>;
        .....
    };
};

```

3.5 USB VBUS Configuration

Rockchip platform's USB VBUS control circuit mainly has three schemes^[3]:

1. Use GPIO to control the power regulator output Vbus 5V supply voltage;
2. Use PMIC (such as RK809/RK817/RK818) to output Vbus 5V supply voltage;
3. After powering on, the hardware directly outputs Vbus 5V supply voltage without the need for software control, generally used for USB Host interfaces;

The RK356x SDK primarily uses the first scheme, taking the OTG and HOST ports of the RK3568 EVB's VBUS configuration as an example:

```

vcc5v0_host: vcc5v0-host-regulator {
    compatible = "regulator-fixed";
    enable-active-high;
    gpio = <&gpio0 RK_PA6 GPIO_ACTIVE_HIGH>;
    pinctrl-names = "default";
    pinctrl-0 = <&vcc5v0_host_en>;
    regulator-name = "vcc5v0_host";
    regulator-always-on; /* Host Vbus is always powered, so the configuration attribute
is regulator-always-on */
};

vcc5v0_otg: vcc5v0-otg-regulator {
    compatible = "regulator-fixed";
    enable-active-high;
    gpio = <&gpio0 RK_PA5 GPIO_ACTIVE_HIGH>;
    pinctrl-names = "default";
    pinctrl-0 = <&vcc5v0_otg_en>;
    regulator-name = "vcc5v0_otg";
};

&pinctrl {
    usb {
        vcc5v0_host_en: vcc5v0-host-en {
            rockchip,pins = <0 RK_PA6 RK_FUNC_GPIO &pcfg_pull_none>;
        };

        vcc5v0_otg_en: vcc5v0-otg-en {
            rockchip,pins = <0 RK_PA5 RK_FUNC_GPIO &pcfg_pull_none>;
        };
    };
};

```

```

        };
};

/* Note: The host controls the vbus attribute as phy-supply */
&u2phy0_host {
    phy-supply = <&vcc5v0_host>;
    status = "okay";
};

/* Note: The otg controls the vbus attribute as vbus-supply */
&u2phy0_otg {
    vbus-supply = <&vcc5v0_otg>;
    status = "okay";
};

&u2phy1_host {
    phy-supply = <&vcc5v0_host>;
    status = "okay";
};

&u2phy1_otg {
    phy-supply = <&vcc5v0_host>;
    status = "okay";
};

```

3.6 Considerations for Linux USB DT Configuration

3.6.1 Important Attributes of USB DT Explanation

3.6.1.1 USB 3.1 Controller DT Properties

1. "snps,dis_rxdet_inp3_quirk": A unique property of the RK356x USB 3.1 controller, used to disable the receiver detection (Rx-detect) function in P3 state. After adding this property, when performing receiver detection, the USB3 PHY state will first switch from P3 back to P2, and after the receiver detection (which takes about 30us) is completed, the PHY state will switch back to P3. This property can resolve compatibility issues with some USB3 drives that cannot be recognized.
2. "snps,dis_u2_susphy_quirk": Configures the USB 3.1 controller to not automatically suspend the USB2 PHY. In scenarios where USB 3.1 is forced to use as USB 2.0, this property must be added; otherwise, it may lead to abnormal USB2 functionality. In other scenarios, adding this property will not affect USB functionality but will increase power consumption in USB autosuspend scenarios.
3. "snps,usb2-lpm-disable": Configures the USB 3.1 controller to disable the USB2 LPM function. In scenarios where USB 3.1 is forced to use as USB 2.0, this property must be added to be compatible with USB drives that do not support USB2 LPM.
4. "usb-role-switch": Only used for Linux-5.10 and when the USB physical interface is a standard Type-C interface (with a PD controller chip), and the dr_mode property must be configured as "otg"; if dr_mode is not in "otg" mode, do not configure "usb-role-switch".
5. "extcon": One of the main functions is to dynamically switch the OTG mode, suitable for hardware

circuit designs with non-standard Type-C interfaces (with a PD controller chip), such as USB Micro interfaces or Type-A interfaces, and in solutions where the controller is configured in "otg" mode, to achieve dynamic switching of the OTG mode.

3.6.1.2 USB2 PHY DT Properties

1. "vbus-supply" and "phy-supply": Both properties are used to control the 5V VBUS output, but they have different usage scenarios. "vbus-supply" is used for the OTG port, supporting dynamic switching of VBUS. "phy-supply" is used for the USB2 HOST port, where VBUS 5V is always on after the system powers up;
2. "rockchip,vbus-always-on": This property is used for OTG interfaces where VBUS is constantly powered. Adding this property can resolve issues with USB Devices (e.g., USB ADB) not being recognized. However, it will also disable the USB2 PHY autosuspend function, increasing the dynamic runtime power consumption of the USB2 PHY;
3. "rockchip,dis-u2-susphy", disables the USB2 PHY driver's ability to dynamically enter suspend mode. This is mainly used in USB 2.0 only solutions to maintain the USB2 PHY output clock to the USB controller.

3.6.1.3 USB 3.0/SATA/QSGMII ComboPHY DT Properties

1. "rockchip,dis-u3otg0-port": Disables the USB3 root port of USB3_OTG0, used for scenarios where USB3_OTG0 is forced to use USB 2.0;
2. "rockchip,dis-u3otg1-port": Disables the USB3 root port of USB3_HOST1, used for scenarios where USB3_HOST1 is forced to use USB 2.0.

4. RK356x USB OTG Mode Switching Commands

The RK356x SDK supports switching the USB 2.0/3.0 OTG to Host mode or Peripheral mode through software methods, independent of the OTG ID pin level or Type-C interface in the USB hardware circuit.

- Linux-4.19/5.10 USB OTG Mode Switching Commands (Legacy)

```
#1.Force host mode
echo host > /sys/devices/platform/fe8a0000.usb2-phy/otg_mode
#2.Force peripheral mode
echo peripheral > /sys/devices/platform/fe8a0000.usb2-phy/otg_mode
#3.Force otg mode
echo otg > /sys/devices/platform/fe8a0000.usb2-phy/otg_mode
```

- Linux-5.10 Kernel New Switching Commands

```
#1.Force host mode
echo host > /sys/kernel/debug/usb/fcc00000.usb/mode
#2.Force peripheral mode
echo device > /sys/kernel/debug/usb/fcc00000.usb/mode
```

Note:

The new switching commands added in the Linux-5.10 kernel can address the usage limitations of the old switching commands (which depend on the correct configuration of the "extcon" attribute in the USB DTS).

5. Type-C Controller Chip Support List

Table 5-1 Type-C Controller Chip Support List

Type-C Controller Chip Model	Linux-4.4	Linux-4.19	Linux-5.10	Description
FUSB302	Supported	Supported	Supported	Most commonly used on RK platforms
ET7301B	Supported	Supported	Supported	Fully compatible with hardware and software of FUSB302 ^{note1}
ET7303	Not Supported	Supported	Supported	Hardware compatible with FUSB302, software driver highly similar to RT1711 ^{note2}
HUSB311	Not Supported	Supported	Supported	Recommended for priority use Hardware compatible with FUSB302, but software driver is not compatible ^{note3}
RT1711H	Not Supported	Supported	Supported	Hardware compatible with FUSB302, software driver highly similar to ET7303 ^{note4}
ANX7411	Not Supported	Not Supported	In Debugging	RK3588 adaptation in progress
WUSB3801	SDK Not Supported, Used in Some Projects	Not Supported	Not Supported	Custom single-wire communication mechanism, high error rate, cannot guarantee stable communication.

note1.

- Linux-4.4 does not support the TCPM software framework, so it can only support FUSB302/ET7301B. The two can be directly replaced without the need to modify hardware or software;
- Linux-4.19/5.10 supports the TCPM software framework and TCPCI protocol, theoretically compatible with all Type-C controller chips designed based on the TCPCI standard (e.g., ET7303/HUSB311/RT1711H);

note2.

- The small package ET7303 (it is understood that the original factory currently does not provide a large package) has been verified on the RK platform. The kernel needs to enable CONFIG_TYPEC_ET7303 separately, and the detailed DTS configuration please refer to sections [Linux-4.19 RK3566 Type-C USB 2.0 + HUSB311 DTS Configuration](#) and [Linux-5.10 RK3566 Type-C USB 2.0 + HUSB311 DTS Configuration](#), because both HUSB311 and ET7303 have I2C device addresses of 0x4E, the DTS configuration is basically the same, only the node name and compatible = "etek,et7303" attribute need to be modified.

note3.

- The kernel needs to enable CONFIG_TYPEC_HUSB311 separately, and the detailed DTS configuration please refer to sections [Linux-4.19 RK3566 Type-C USB 2.0 + HUSB311 DTS Configuration](#) and [Linux-5.10 RK3566 Type-C USB 2.0 + HUSB311 DTS Configuration](#).

note4.

- RT1711H is hardware compatible with FUSB302/ET7303, and the software driver is highly similar to ET7303. The kernel needs to enable CONFIG_TYPEC_RT1711H separately, and the DTS configuration is basically the same as HUSB311/ET7303, with I2C device addresses both being 0x4E, only the node name and compatible = "richtek,rt1711h" attribute need to be modified.

6. References

1. **Universal Serial Bus Type-C Cable and Connector Specification**
2. **Rockchip Developer Guide USB CN**
3. **Rockchip RK3399 Developer Guide USB CN**
4. **kernel/Documentation/devicetree/bindings/usb/generic.txt**
5. **kernel/Documentation/devicetree/bindings/usb/dwc3.txt**
6. **kernel/Documentation/devicetree/bindings/usb/rockchip,dwc3.txt**
7. **kernel/Documentation/devicetree/bindings/usb/usb-ehci.txt**
8. **kernel/Documentation/devicetree/bindings/usb/usb-ohci.txt**
9. **kernel/Documentation/devicetree/bindings/phy/phy-rockchip-naneng-combphy.txt**
10. **kernel/Documentation/devicetree/bindings/phy/phy-rockchip-inno-usb2.txt**